

딥러닝 기초

고려대학교 석준희

*ChatGPT: Optimizing
Language Models
for Dialogue*

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible to challenge incorrect premises, and request more information. ChatGPT is a sibling model to GPT-3, which follows an instruction-based paradigm.

목차

- 이진분류
- 퍼셉트론 (Perceptron)
- 다계층 퍼셉트론 (MLP: Multi-Layer Perceptron)
- 다양한 학습 기법
- 텐서플로우

답러닝 기초

이진분류



분류 문제의 표현

- 회귀문제 (Regression): Y 가 연속형 변수일 때
- 분류문제 (Classification): Y 가 범주형 변수일 때
- 클래스 혹은 범주를 숫자로 표현이 가능할까?
 - $Y = A \text{ or } B \rightarrow Y = 1 \text{ or } 2$: 가능함
 - $Y = A, B \text{ or } C \rightarrow Y = 1, 2 \text{ or } 3$: 불가능함
 - 기본적으로 회귀문제로 치환하여 해결하는 것이 불가능
- 많은 분류의 문제들이 클래스 Y 를 직접 예측하기 보다는 Y 가 특정 클래스일 확률 $\Pr[Y = k|X]$ 를 예측하고자 함

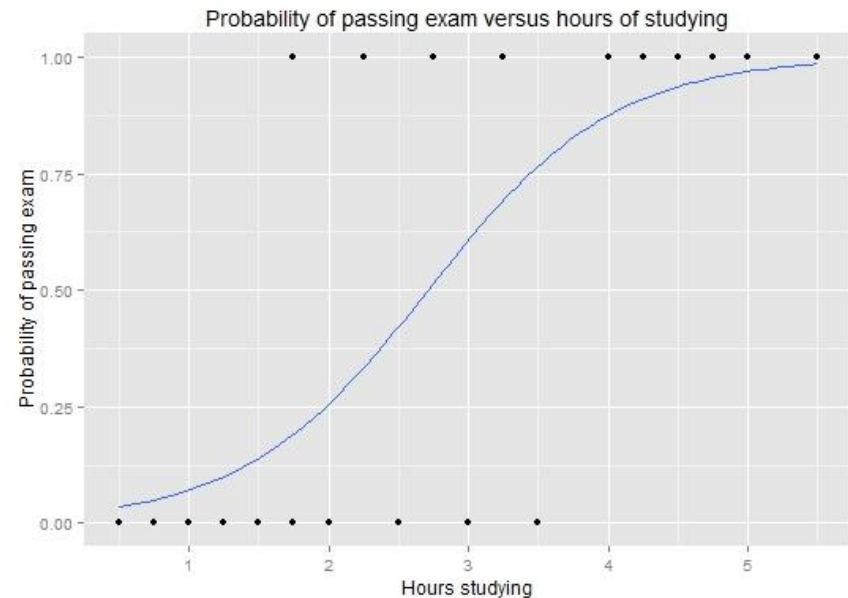
$$\Pr[Y = k|X] \sim f(X)$$



이진 분류 (Binary Classification)

- 이진 분류: 범주/클래스가 두 개인 경우
- 예제: 공부시간 vs. 시험합격
 - Hours : 공부시간 (숫자)
 - Pass: 합격 여부 (합격 or 불합격)

	공부시간	합격여부	$\Pr[Y=1 X]$
1	0.50	불합격	0
2	3.30	합격	1
3	1.75	합격	1
4	3.00	불합격	0



https://en.wikipedia.org/wiki/Logistic_regression



로지스틱 회귀 (Logistic Regression)

- 클래스에 대한 확률을 시그모이드 함수를 이용하여 모델링
- 이진분류 ($Y = 1$ or 0)에 대하여

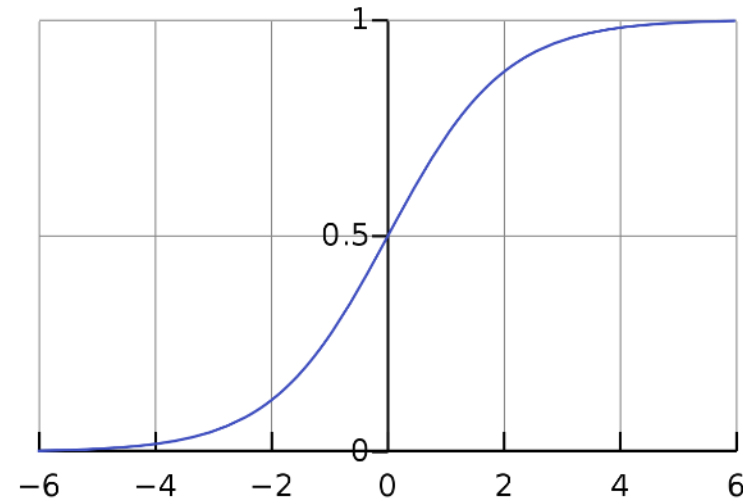
$$\Pr[Y = 1|X] = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

$$\Pr[Y = 0|X] = \frac{1}{1 + e^{\beta_0 + \beta_1 X}} = \frac{e^{-(\beta_0 + \beta_1 X)}}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

$$\log \left(\frac{\Pr[Y = 1|X]}{\Pr[Y = 0|X]} \right) = \beta_0 + \beta_1 X$$

승수(odds)

로지트(logit)



https://en.wikipedia.org/wiki/Sigmoid_function



파라미터의 추정

- 우도 (Likelihood): 어떤 모델을 가정했을 때 현재 데이터를 관측할 확률
- 로지스틱 회귀에서의 우도

$$\text{모델: } \Pr[Y = 1|X] = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}, \quad \Pr[Y = 0|X] = \frac{1}{1 + e^{\beta_0 + \beta_1 X}}$$

$$\text{우도: } l(\beta_0, \beta_1) = \prod_{i: y_i=0} \frac{1}{1 + e^{\beta_0 + \beta_1 x_i}} \prod_{i: y_i=1} \frac{e^{\beta_0 + \beta_1 x_i}}{1 + e^{\beta_0 + \beta_1 x_i}}$$

- 예제

	공부시간	합격여부	$\Pr[Y=1 X]$
1	0.50	불합격	0
2	3.30	합격	1
3	1.75	합격	1
4	3.00	불합격	0



파라미터의 추정

- **최대우도법 (Maximum Likelihood)**

- 왜 하필 우리는 이 데이터를 관측하고 있을까? 그것은 이 데이터를 관측할 확률이 제일 높기 때문이다!
- 우도를 제일 크게 만드는 파라미터가 진짜 파라미터

$$l(\beta_0, \beta_1) = \prod_{i:y_i=0} \frac{1}{1 + e^{\beta_0 + \beta_1 x_i}} \prod_{i:y_i=1} \frac{e^{\beta_0 + \beta_1 x_i}}{1 + e^{\beta_0 + \beta_1 x_i}}$$

- 보통 우도의 미분을 통해 파라미터를 찾는 것은 계산이 어려움
- 우도의 로그값(Log-likelihood)을 최대화하는 파라미터를 찾음
 - 로그 함수의 특성상 $f(x)$ 를 최대화 하는 것은 $\log(f(x))$ 를 최대화 하는 것과 같음

$$\hat{\beta}_0, \hat{\beta}_1 = \operatorname{argmax}(l) = \operatorname{argmax}(\log l)$$

$$\frac{\partial \log l}{\partial \beta_0} = 0 \quad \frac{\partial \log l}{\partial \beta_1} = 0 \quad \Rightarrow \quad \hat{\beta}_0, \hat{\beta}_1$$



파라미터의 추정

- 크로스엔트로피(cross-entropy) 손실 함수

$$\text{Let } p(x) = \Pr[y = 1|x] = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

$$l = \prod_{i:y_i=1} p(x_i) \prod_{i:y_i=0} (1 - p(x_i)) = \prod_i p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$$

$$Loss = -\log l = -\sum_i [y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i))]$$

Cross-entropy loss

- Maximizing the likelihood of logistic regression is equivalent to minimizing the cross-entropy loss.



파라미터의 추정

- Another view of maximizing likelihood
 - The previous cross-entropy is when $y_i \in \{0,1\}$, but in modern machine learning sometimes $y_i \in \{-1,1\}$ is preferred.

$$\Pr[y = 1|x] = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}} = \frac{1}{1 + e^{-y(\beta_0 + \beta_1 x)}}$$

$$\Pr[y = -1|x] = \frac{1}{1 + e^{\beta_0 + \beta_1 x}} = \frac{1}{1 + e^{-y(\beta_0 + \beta_1 x)}}$$

$$l = -\prod_i \frac{1}{1 + e^{-y_i(\beta_0 + \beta_1 x_i)}}$$

$$Loss = -\log l = \sum_i \log(1 + e^{-y_i(\beta_0 + \beta_1 x_i)})$$

Logistic loss

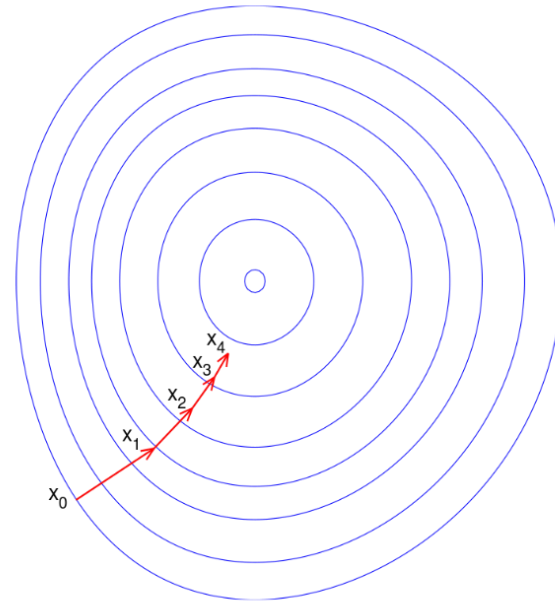


파라미터의 추정

- 수치적 해법
 - 우도에 대한 미분을 수식으로 풀 수 없어 수치적으로 풀어야 함
 - 임의의 파라미터에서 시작하여 경사(미분값)을 따라 업데이트해 가는 방식
 - 예: Newton-Raphson 법, 경사하강법 등

$\beta^{(0)}$: random position

$$\beta^{(i+1)} = \beta^{(i)} - \lambda \frac{\partial L(\beta^{(i)})}{\partial \beta}$$



https://en.wikipedia.org/wiki/Gradient_descent

일반적인 로지스틱 회귀 (Multiple Logistic Regression)

- 하나 이상의 독립 변수에 대해서 선형 회귀와 같이 확장

$$\log \left(\frac{\Pr[Y = 1|X]}{\Pr[Y = 0|X]} \right) = \beta_0 + \beta_1 X$$

$$\log \left(\frac{\Pr[Y = 1|X]}{\Pr[Y = 0|X]} \right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

$$\Pr[Y = 1|X] = \frac{e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p}}$$

- 선형회귀에서 사용되었던 다양한 기법을 바로 적용 가능
 - 가변수를 이용한 범주형 변수의 표현
 - 상호작용의 고려
 - 고차원 변수의 도입
 - 모델의 복잡성에 대한 고려도 동일



분류 모델의 평가

- 정확도 (Accuracy): 각 클래스를 정확히 맞춘 비율

$$Acc = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{f}(x_i))$$

- 혼동행렬 (Confusion Matrix): 실제값과 예측값을 행렬의 형태로 표현

		truth			
		A	B	C	D
predicted	A	70	10	15	5
	B	8	67	20	5
	C	0	11	88	1
	D	4	10	14	72



이진 분류 모델의 평가

- 두 개의 클래스에 대한 2x2 혼동 행렬
 - 양성 (Positive): 우리가 찾고 싶은 클래스 (e.g. 질병, 부도 등)
 - 음성 (Negative): 다른 하나의 클래스 (e.g. 정상, 신용 등)

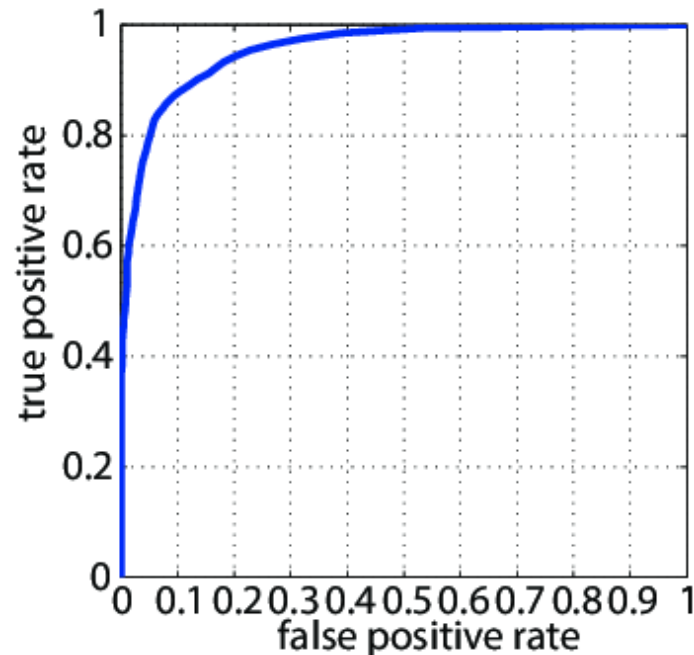
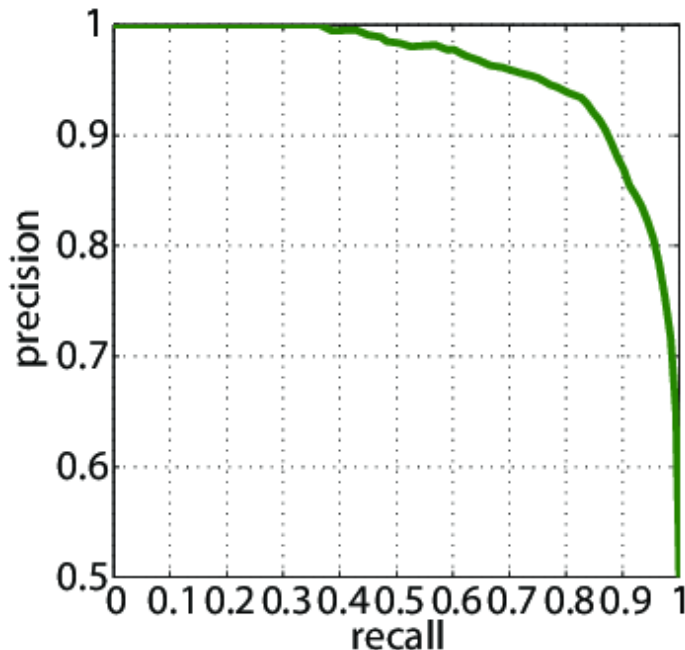
	Truly Positive	Truly Negative
Predicted Positive	True Positive (TP)	False Positive (FP)
Predicted Negative	False Negative (FN)	True Negative (TN)

- **정확도(Accuracy)** = $(TP+TN)/(TP+FP+FN+TN)$
- **재현율(Recall, True Positive Rate, Sensitivity)** = $TP/(TP+FN)$
- **정밀도(Precision, Positive Predictive Value)** = $TP/(TP+FP)$
- **위양성률(False Positive Rate)** = $FP/(FP+TN)$
- **F_1 Score** = $2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \left(\frac{P^{-1} + R^{-1}}{2} \right)^{-1} = \frac{TP}{TP + (FP + FN)/2}$



이진 분류 모델의 평가

- **Precision-Recall curve:** recall이 증가함에 따라 precision은 대체로 감소하지만 증가하기도 함
 - **mAP (mean Average Precision):** 단조감소화한 PR 커브의 면적
- **Receive-operating character (ROC) curve:** FPR이 증가함에 따라 TPR은 항상 같거나 증가
 - **AUC (area under curve):** ROC 커브의 면적





예제: 부도 예측

- Output: default
- Input: balance, income
- Data description

	Default: Yes	Default: No	
Train	174	4,826	5,000
Test	159	4,841	5,000

- Model: default ~ balance + income

Train Set	Cond. Yes	Cond. No
Pred. Yes	59	19
Pred. No	115	4,807

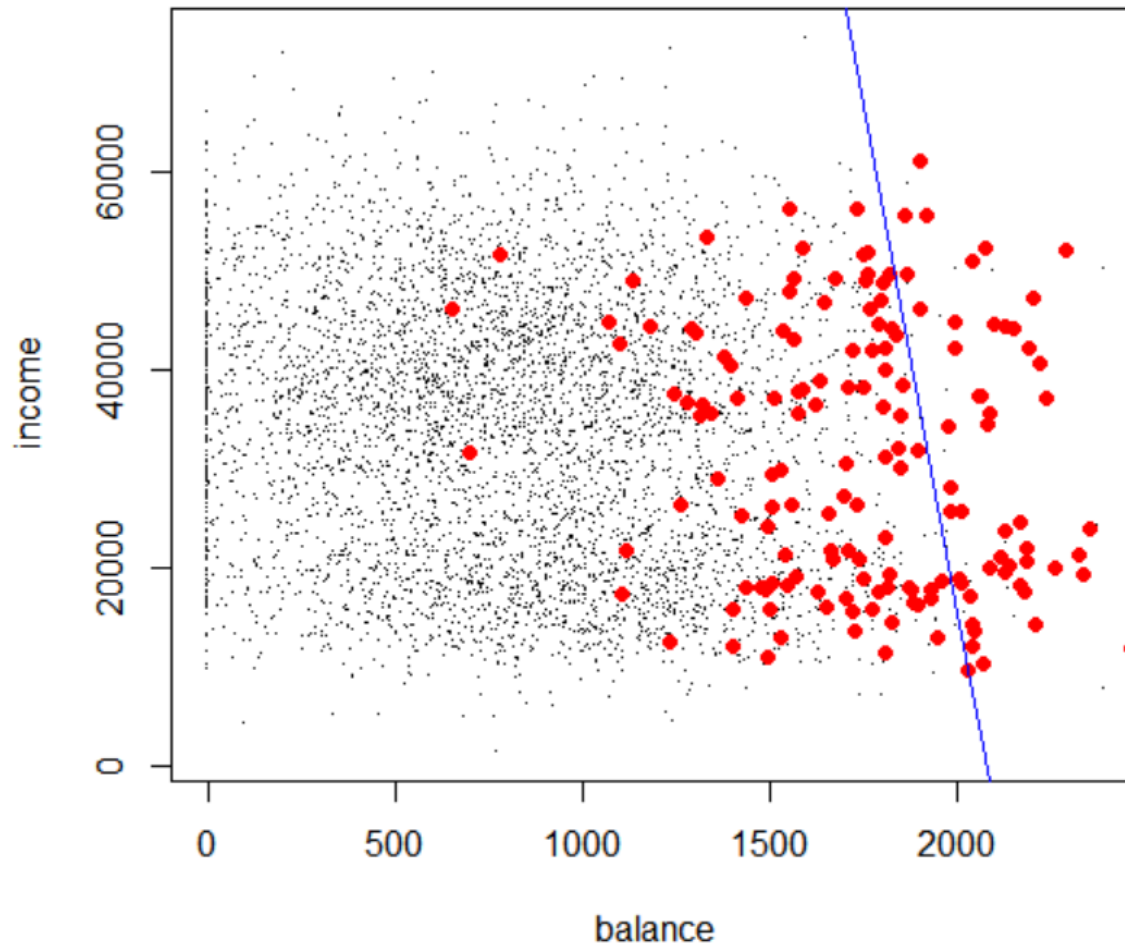
Test Set	Cond. Yes	Cond. No
Pred. Yes	48	22
Pred. No	111	4,819

	Acc	Recall	Precision	FPR	F1
Train	0.97	0.34	0.76	0.00	0.47
Test	0.97	0.30	0.69	0.00	0.42



예제: 부도 예측

- 모델의 판단 경계 (decision boundary): $\text{default} \sim \text{balance} + \text{income}$
 - 판단의 경계는 항상 선형 (linear): 선형 모델!!



딥러닝 기초

퍼셉트론



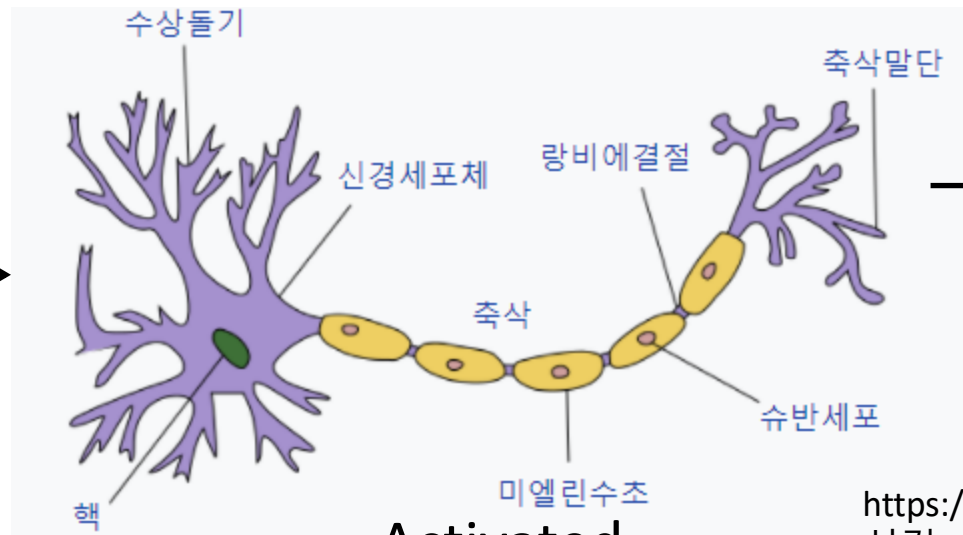
인공신경망 (ANN: Artificial Neural Network)

- 인공지능의 목표
 - 인간과 같이 지능적이 판단을 할 수 있는 기계장치 혹은 컴퓨터 프로그램을 만드는 것
- 지능의 문제
 - (유형 1) 1억 개의 숫자에 대한 중간값 구하기
 - (유형 2) 남자와 여자의 구별
- 인간은 쉽게 풀지만 컴퓨터는 풀지 못하는 유형의 문제가 존재
 - 안면인식, 언어처리, 음성인식 등
 - 이들 문제를 해결하는 것이 오랫동안 인공지능의 과제였음
- 인공신경망의 도입
 - 위의 문제를 푸는 구체적인 알고리즘을 고안하는 대신, 인간의 뇌를 닮은 수학적 모델을 만들어서 해결하자!



퍼셉트론 (Perceptron)

Stimulated →



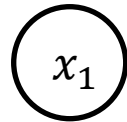
→ Chemicals

https://ko.wikipedia.org/wiki/신경_세포

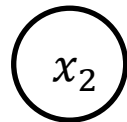
Input



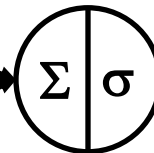
b



w_1



w_2



Output

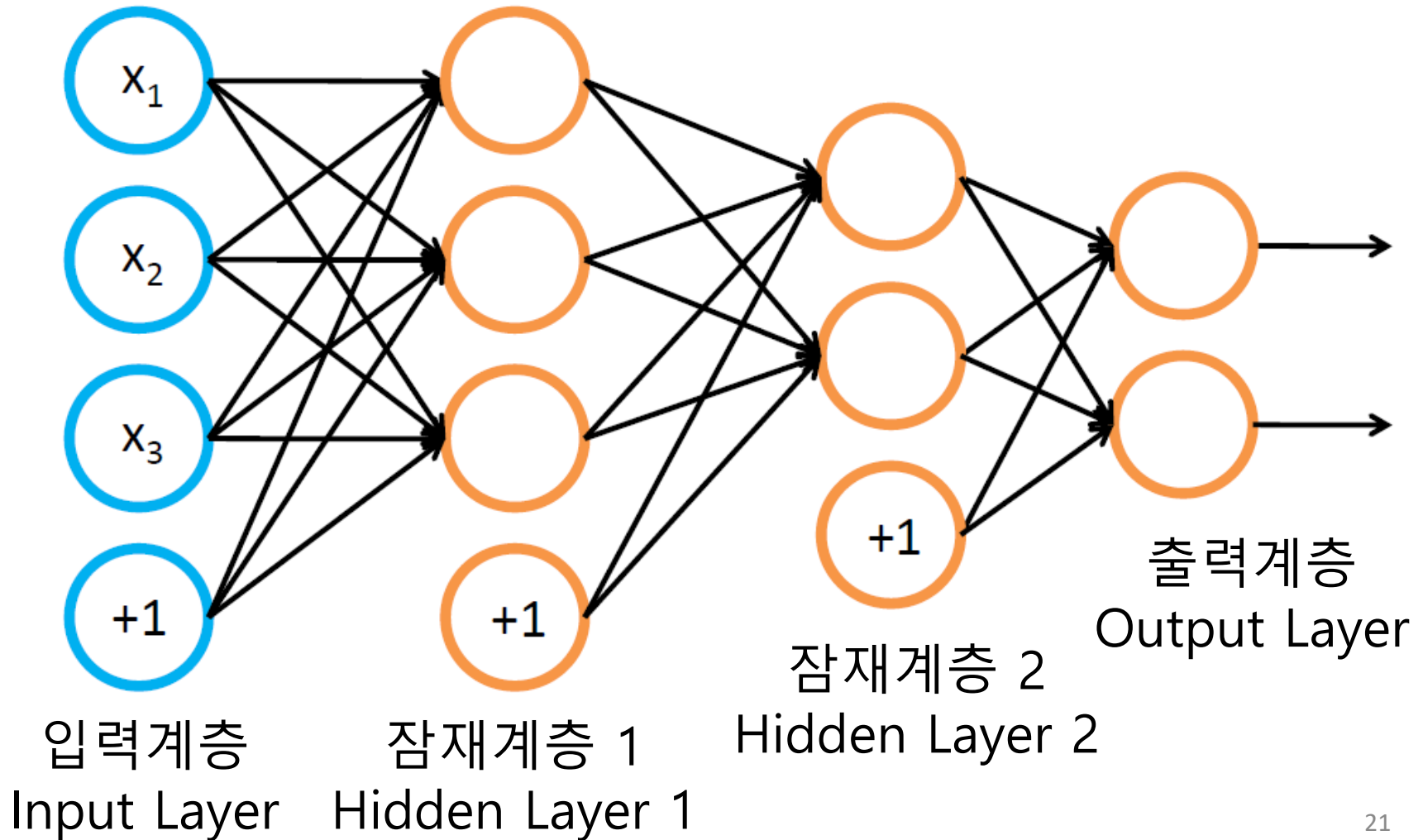
Activation Function

$$y = \sigma \left(\sum_j w_j x_j + b \right) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$



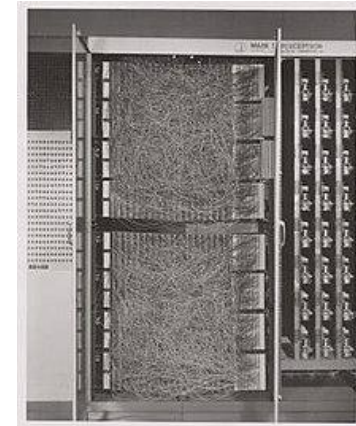
인공신경망 (ANN: Artificial Neural Network)

- 퍼셉트론의 네트워크



초기의 퍼셉트론

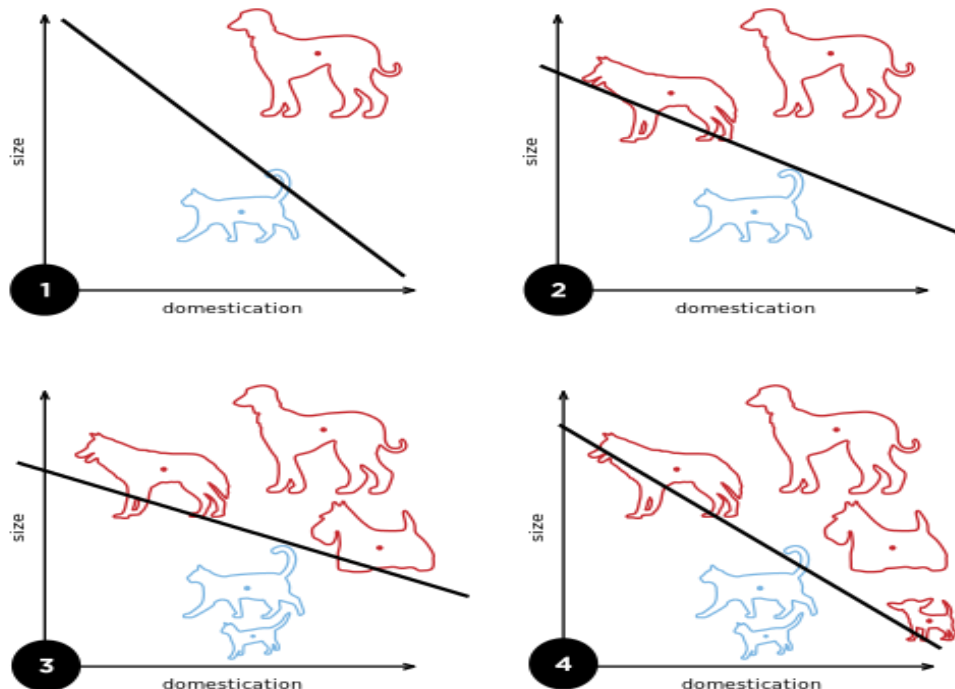
- 퍼셉트론은 1958년에 처음 발명되었음
 - 강력한 CPU의 부재: 계산이 간단하여야 함
 - 저장장치의 부재: 훈련 데이터가 저장될 수 없음
- 초기 퍼셉트론 알고리즘
 - 스텝 활성화 함수의 사용
 - 온라인 학습: 샘플이 하나씩 사용



Mark I
Perceptron
Machine

using electric
motor
to update
weights

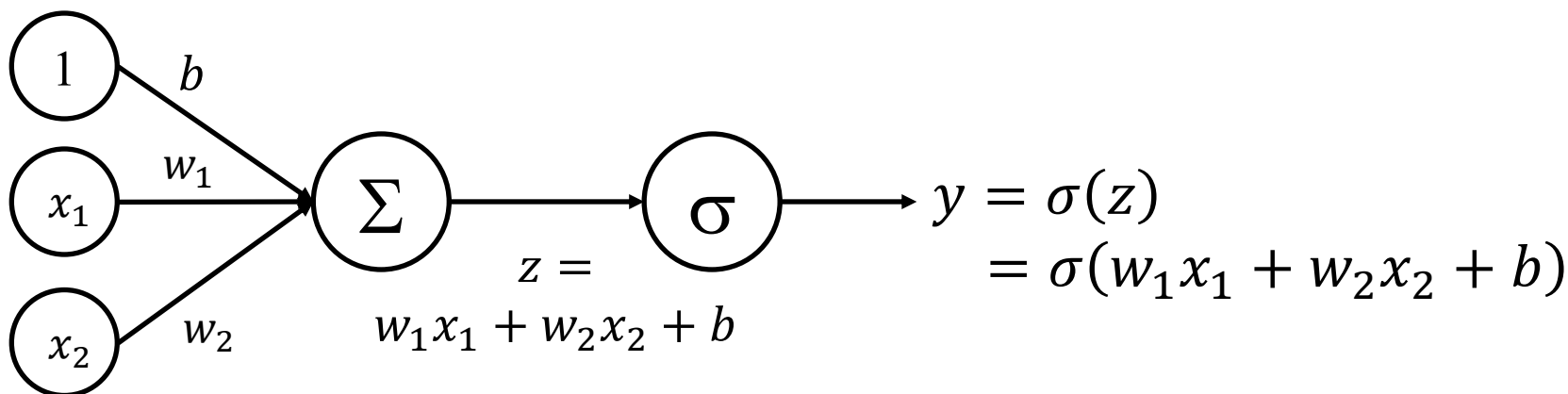
<https://en.wikipedia.org/wiki/Perceptron>



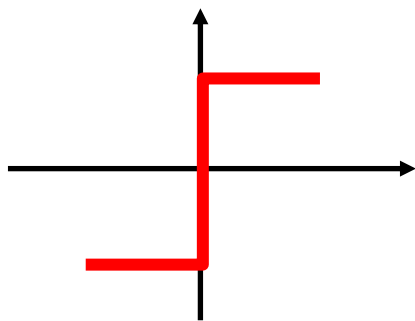


현대의 퍼셉트론

- 현대적 관점에서 퍼셉트론은 선형모델을 일반화한 모델로 볼 수 있음
 - 활성화 함수에 따라 다양한 모델을 표현하는 것이 가능

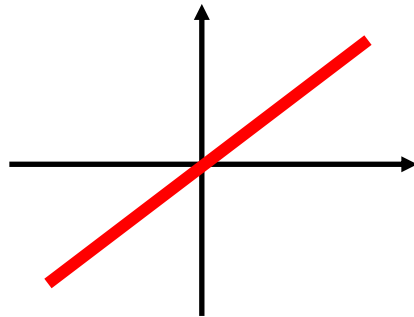


Step Function



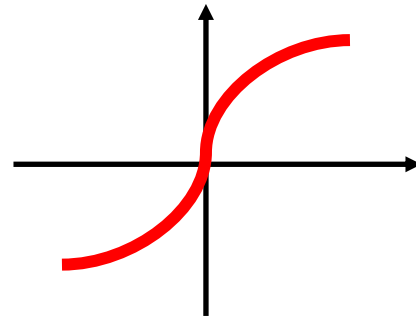
The very first perceptron uses it because it is easy to implement in hardware at that time (1960)

Identity



$y = \sigma(z) = z$
 $= w_1x_1 + w_2x_2 + b$
Linear Regression

Sigmoid



$y = \sigma(z) = \frac{1}{1 + e^{-z}}$
Logistic Regression²³



파라미터의 학습

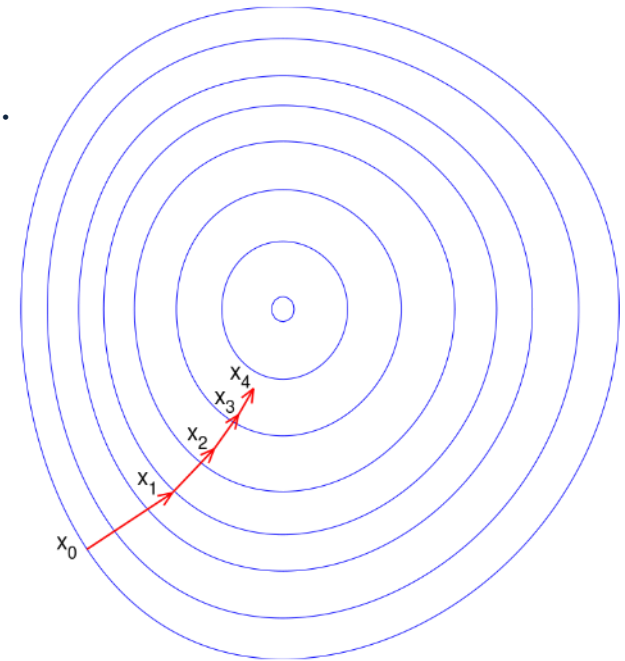
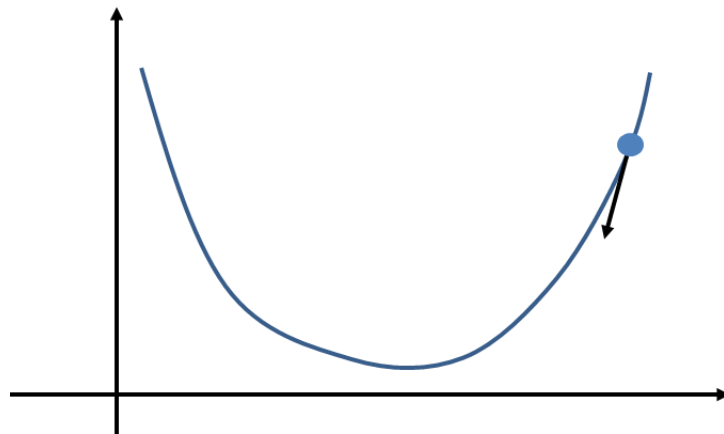
- 훈련데이터로 부터 어떻게 모델의 파라미터를 학습하는가?
 - 전통적으로 서로 다른 모델은 서로 다른 알고리즘을 사용
 - 퍼셉트론은 일반적인 모델이기 때문에 모든 활성화 함수에 적용가능한 일반적인 방법이 필요
 - 손실 함수의 최소화
 - 해결하고자 하는 문제와 활성화 함수에 따라 손실 함수를 정의
 - 손실을 최소화하는 최적의 파라미터를 탐색
 - 경사하강법과 같은 일반적인 최적화 기법을 사용
 - 예제
 - 선형함수(identity activation): 제곱손실 함수로 표현 (최소자승법과 동일)
- $$L(w, b) = \sum_{i=1}^n (y_i - f(x_i))^2 = \sum_{i=1}^n (y_i - wx_i - b)^2$$
- 시그모이드(sigmoid activation): 로지스틱 손실 함수 (최대우도법과 동일)

$$L(w, b) = \sum_{i=1}^n \log(1 + e^{-y_i f(x_i)}) = \sum_{i=1}^n \log(1 + e^{-y_i (wx_i + b)})$$



경사하강법 (Gradient Descent)

- 일반적인 최적화 문제를 풀고 싶음: $\min_{w,b} L(w, b)$.
- 경사하강법(Gradient descent)이 적용 가능



https://en.wikipedia.org/wiki/Gradient_descent

- 대략적 알고리즘
 - (1) 파라미터 초기화: $w^{(0)} = w_0, b^{(0)} = b_0$ (usually 0's)
 - (2) 파라미터 업데이트

$$w^{(k+1)} = w^{(k)} - \lambda \frac{\partial L}{\partial w}$$

- (3) 수렴할 때까지 (2)를 반복

$$b^{(k+1)} = b^{(k)} - \lambda \frac{\partial L}{\partial b}$$

경사도
gradient

학습률
learning rate



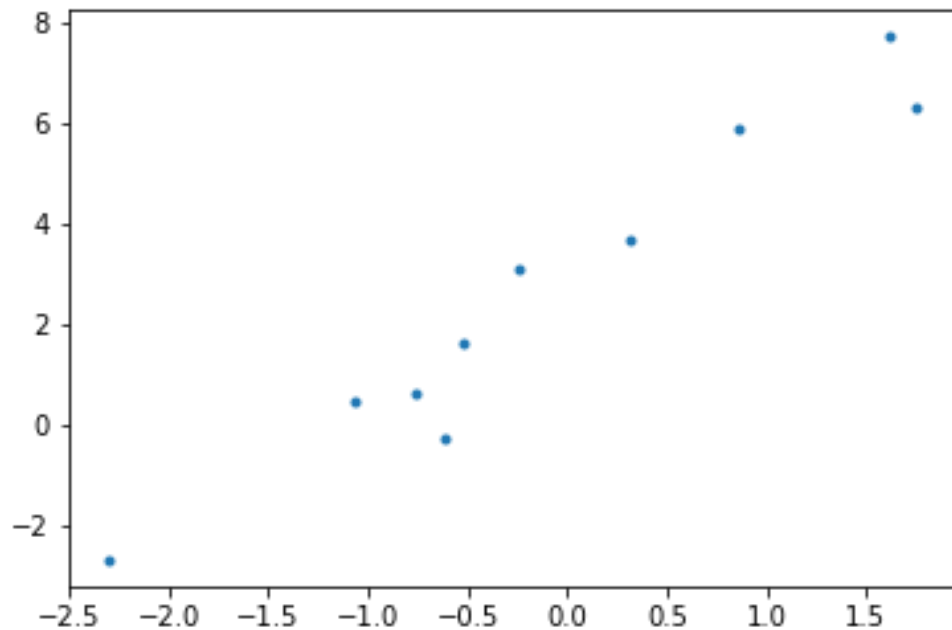
경사하강법 (Gradient Descent)

- 예제: 제곱 손실을 이용하는 선형 활성화 함수

$$Y \approx wX + b \rightarrow L(w, b) = \sum_{i=1}^n (y_i - wx_i - b)^2$$

$$\frac{\partial L}{\partial w} = -2 \sum_{i=1}^n x_i (y_i - wx_i - b) \quad \frac{\partial L}{\partial b} = -\sum_{i=1}^n (y_i - wx_i - b)$$

	x	y
0	1.624345	7.710799
1	-0.611756	-0.283654
2	-0.528172	1.621239
3	-1.072969	0.470008
4	0.865408	5.864585
5	-2.301539	-2.702969
6	1.744812	6.317195
7	-0.761207	0.599728
8	0.319039	3.680292
9	-0.249370	3.084074



최소자승법을 이용한 선형회귀에 따르면, $w=2.55$ and $b=2.88$

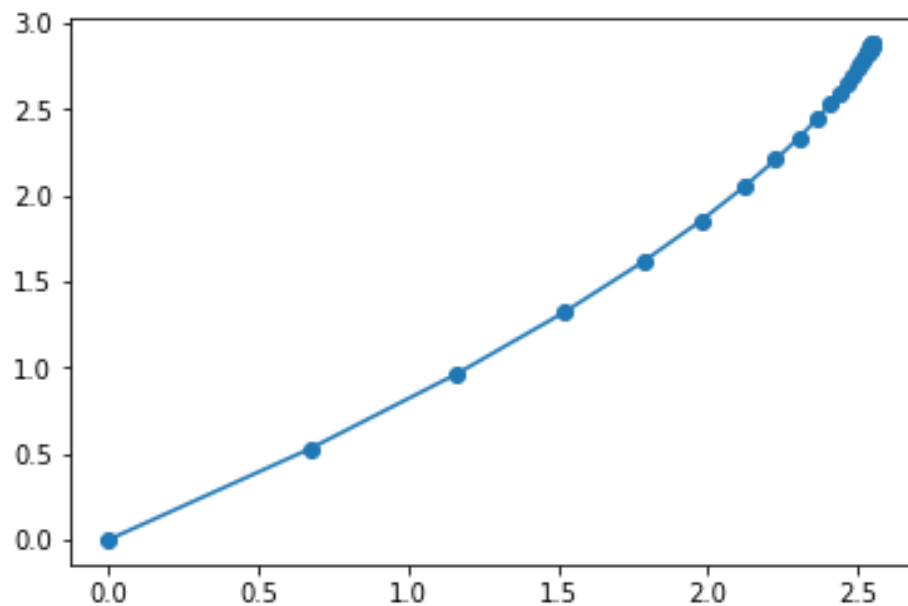


경사하강법 (Gradient Descent)

- 예제: 제곱 손실을 이용하는 선형 활성화 함수

	w	b	diff
0	0.000000	0.000000	0.000000
1	0.672101	0.527226	0.364844
2	1.162537	0.962064	0.214806
3	1.521384	1.319463	0.128253
4	1.784711	1.612354	0.077563
5	1.978540	1.851783	0.047448
6	2.121675	2.047092	0.029317
7	2.227734	2.206120	0.018269
8	2.306600	2.335403	0.011467
9	2.365458	2.440361	0.007240
10	2.409550	2.525471	0.004594

b	diff
842245	5.512791e-05
850350	3.560605e-05
856880	2.300401e-05
862139	1.486562e-05
866374	9.608168e-06
869785	6.210963e-06
872530	4.015363e-06
27	2.547892 2.874740 2.596138e-06
28	2.548331 2.876519 1.678648e-06
29	2.548680 2.877951 1.085461e-06
30	2.548956 2.879103 7.019176e-07



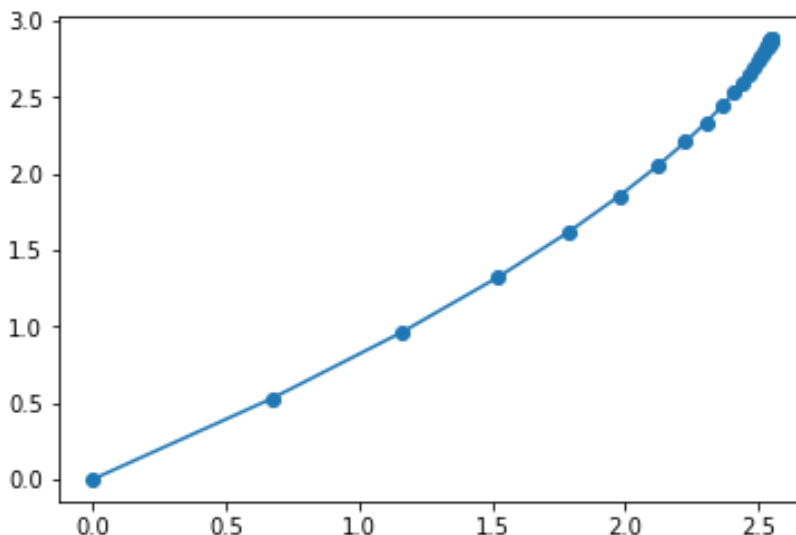


확률적 경사하강법 (SGD: Stochastic Gradient Descent)

- 일반적 경사하강법은 경사도를 계산하기 위해 모든 데이터를 사용하는데 이는 계산량이 많음
- **확률적 경사하강법 (SGD: Stochastic gradient descent):** 하나의 데이터를 이용하여 근사적으로 경사를 계산하는 방법
 - 손실을 바로 줄이지는 못하지만 더 빠르게 계산이 가능
 - 각 스텝은 부정확하지만 결국은 비슷한 곳으로 수렴

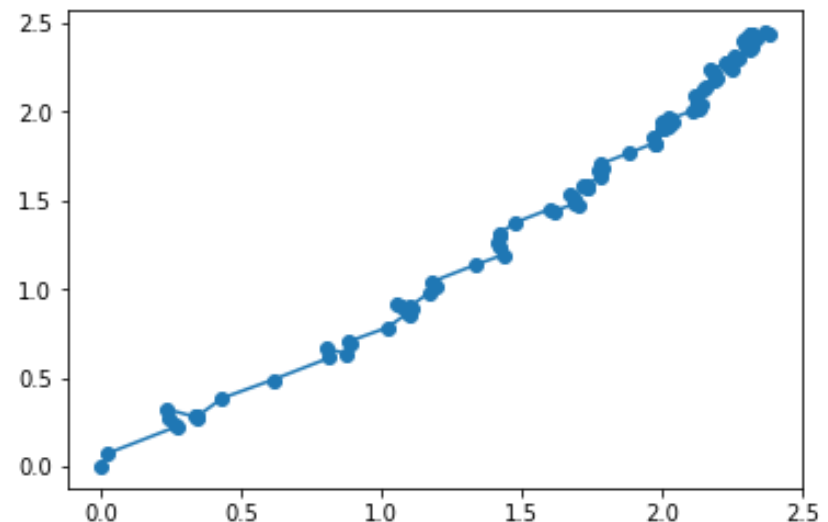
Ordinary Gradient Descent

$$\frac{\partial L}{\partial w} = -2 \sum_{i=1}^n x_i (y_i - wx_i - b)$$



Stochastic Gradient Descent

$$\frac{\partial L}{\partial w} \approx -2x_i(y_i - wx_i - b)$$





확률적 경사하강법 (SGD: Stochastic Gradient Descent)

- Example: sigmoid activation and logistic loss, $y \in \{-1, 1\}$

$$L(w, b) = \sum_{i=1}^n \log(1 + e^{-y_i(wx_i+b)})$$

$$\frac{\partial L}{\partial w} = - \sum_{i=1}^n \frac{x_i y_i}{1 + e^{y_i(wx_i+b)}}$$

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \frac{y_i}{1 + e^{y_i(wx_i+b)}}$$

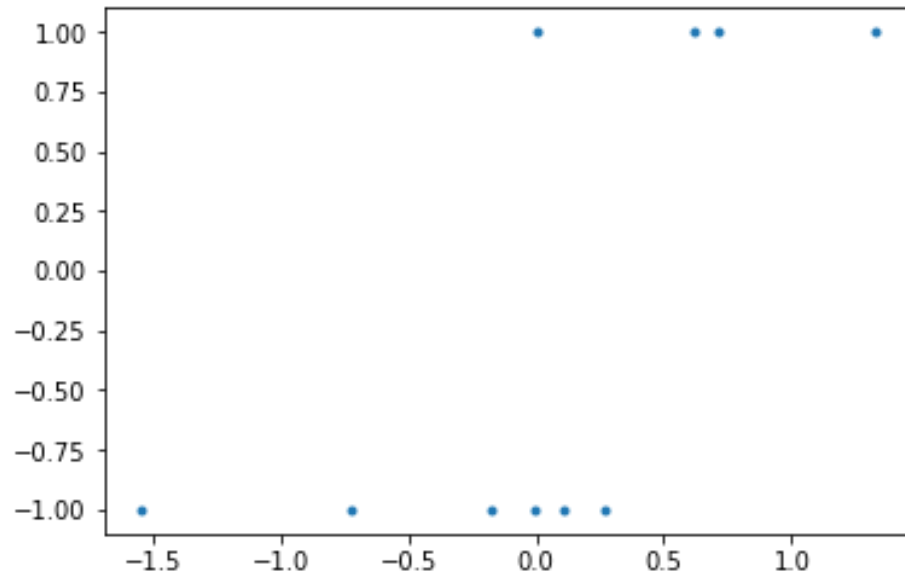
Usual Gradient Descent

$$\frac{\partial L}{\partial w} = - \frac{x_i y_i}{1 + e^{y_i(wx_i+b)}}$$

$$\frac{\partial L}{\partial b} = - \frac{y_i}{1 + e^{y_i(wx_i+b)}}$$

Stochastic Gradient Descent

	x	y
0	1.331587	1
1	0.715279	1
2	-1.545400	-1
3	-0.008384	-1
4	0.621336	1
5	-0.720086	-1
6	0.265512	-1
7	0.108549	-1
8	0.004291	1
9	-0.174600	-1



From logistic regression, $w=5.55$ and $b=-1.52$



확률적 경사하강법 (SGD: Stochastic Gradient Descent)

- Example: sigmoid activation and logistic loss, $y \in \{-1, 1\}$

$$L(w, b) = \sum_{i=1}^n \log(1 + e^{-y_i(wx_i + b)})$$

$$\frac{\partial L}{\partial w} = - \sum_{i=1}^n \frac{x_i y_i}{1 + e^{y_i(wx_i + b)}}$$

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \frac{y_i}{1 + e^{y_i(wx_i + b)}}$$

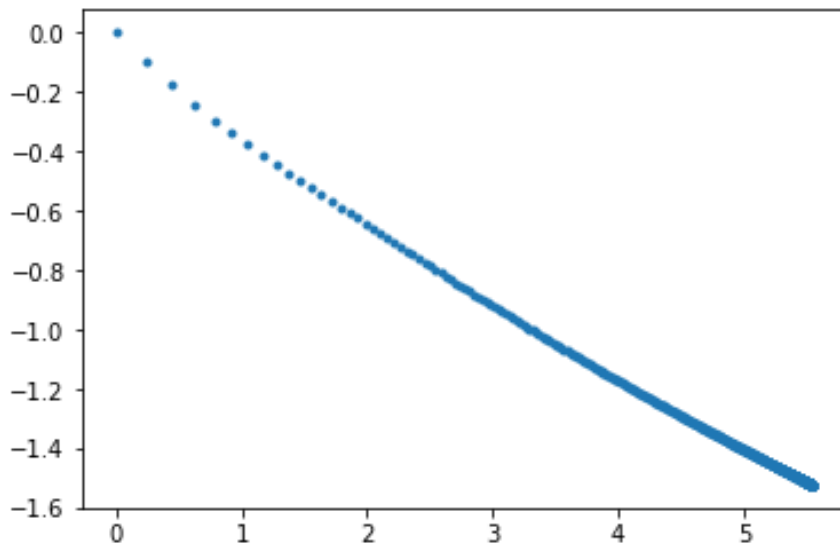
Usual Gradient Descent

$$\frac{\partial L}{\partial w} = - \frac{x_i y_i}{1 + e^{y_i(wx_i + b)}}$$

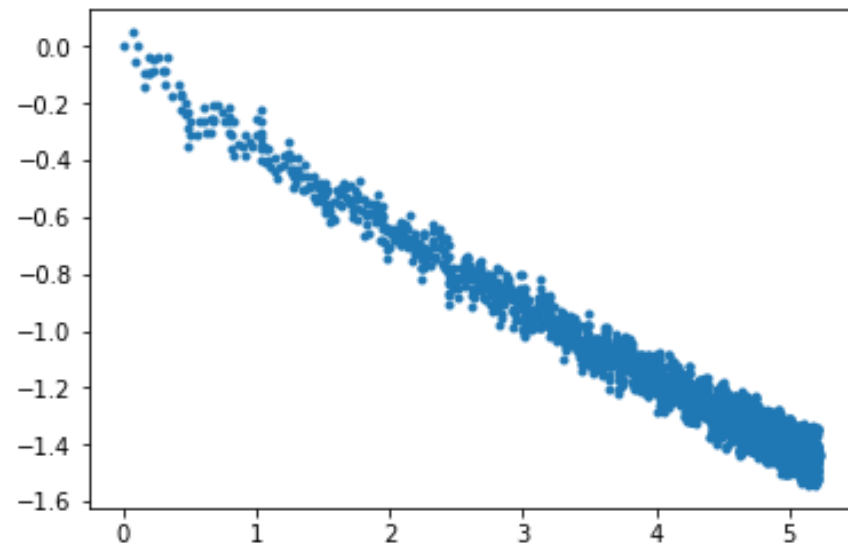
$$\frac{\partial L}{\partial b} = - \frac{y_i}{1 + e^{y_i(wx_i + b)}}$$

Stochastic Gradient Descent

Ordinary Gradient Descent
using 1,022 x 10 samples



Stochastic Gradient Descent
Using 2,830 samples





미니배치 (Mini-batch)

- 배치(batch): 훈련에 사용되는 한 세트의 데이터 (예: 훈련집합)
- 일반적 경사하강법에서는 파라미터 업데이트 한 번에 전체 배치가 사용
- 확률적 경사하강법에서는 하나의 데이터로 업데이트
- 미니배치를 이용한 확률적 경사하강법
 - 일반적인 경사하강법과 확률적 경사하강법의 중간쯤 위치
 - 전체 훈련집합을 고정된 크기의 작은 집합 여러 개로 분할
 - 미니배치(mini-batch): 각각의 작은 집합
 - 파라미터가 하나의 미니배치로 업데이트
- 대략적 알고리즘
 - (1) 전체 훈련집합을 여러 개의 미니배치로 분할, $B_1, B_2, \dots, B_N$
 - (2) 각 미니 배치에 대하여(For b from 1 to n),

$$\frac{\partial L}{\partial w} = \sum_{i \in B_i} \frac{\partial L_i}{\partial w} \quad \text{e.g.} \quad \frac{\partial L}{\partial w} = -2 \sum_{i \in B_i} x_i (y_i - wx_i - b)$$
 - (3) 수렴할 때까지 (2)를 반복



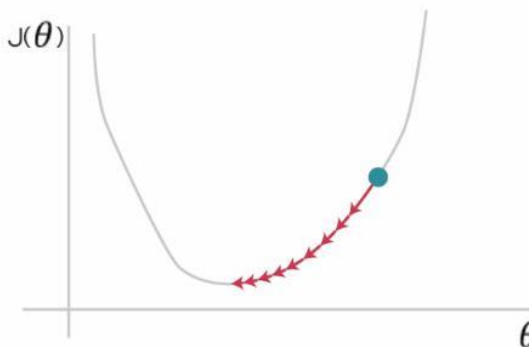
학습률 (Learning Rate)

- 학습률(Learning rate)은 얼마나 빨리 파라미터를 업데이트 할지를 결정

$$w^{(k+1)} = w^{(k)} - \lambda \frac{\partial L}{\partial w}$$

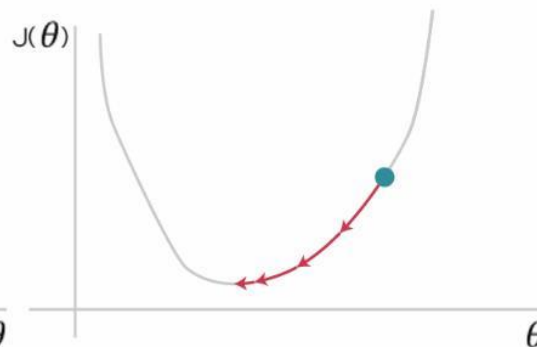
- 작은 λ : 느리지만 안정적인 학습
- 큰 λ : 빠르지만 불안정한 학습 (수렴하지 않을 수도 있음)

Too low



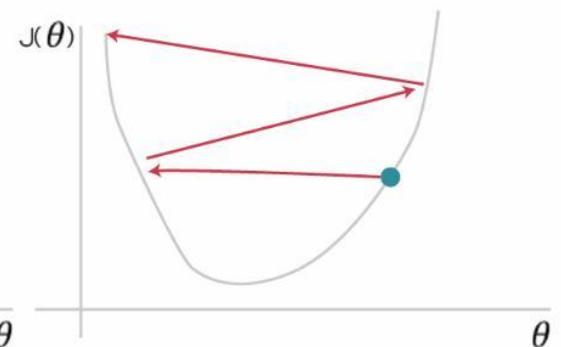
A small learning rate requires many updates before reaching the minimum point

Just right



The optional learning rate swiftly reaches the minimum point

Too high

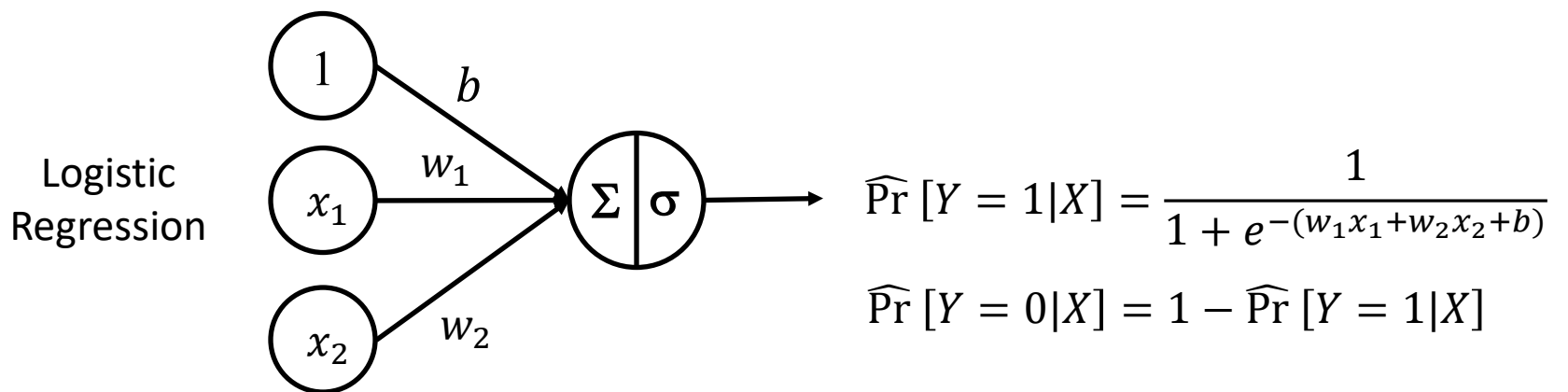


Too large of a learning rate cause drastic updates which lead to divergent behaviors



이진 분류 퍼셉트론

- 이진분류 퍼셉트론 모델: 로지스틱 회귀를 모델링
 - 시그모이드 활성화 함수: $\sigma(z) = \frac{1}{1+e^{-z}}$
 - 손실함수: 크로스 엔트로피 손실 함수



$$L(\mathbf{w}, b) = -\sum_i [y_i \log(p) + (1 - y_i) \log(1 - p)]$$

$$p = \frac{1}{1 + e^{-(w_1x_1 + w_2x_2 + b)}}$$



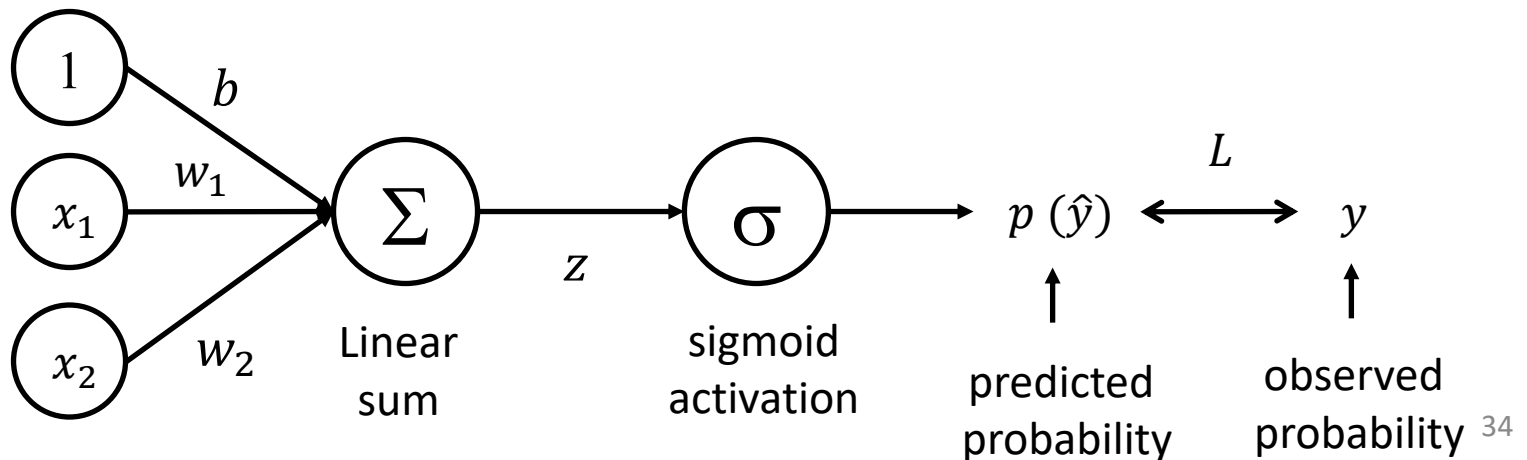
이진 분류 퍼셉트론

$$L = -(y \log(p) + (1 - y) \log(1 - p)) \quad \longleftarrow \quad \text{Loss for one sample}$$

$$p = \frac{1}{1 + e^{-z}} \quad z = w_1 x_1 + w_2 x_2 + b$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial p} \frac{\partial p}{\partial z} \frac{\partial z}{\partial w_1} = \left[-\frac{y}{p} + \frac{1-y}{1-p} \right] \cdot p(1-p) \cdot x_1 = (p - y)x_1$$

$$\frac{\partial L}{\partial w_1} = (p - y)x_1 \quad \frac{\partial L}{\partial w_2} = (p - y)x_2 \quad \frac{\partial L}{\partial b} = p - y$$





One-hot Encoding

- One-hot encoding: 기계학습에서 범주형 변수를 표현하는 기본적인 방법
- 예제: 자연어 문장의 표현, "I love you, you love me".

	I	Love	You	You	Love	Me
I	1	0	0	0	0	0
Love	0	1	0	0	1	0
You	0	0	1	1	0	0
Me	0	0	0	0	0	1

- 좀 더 일반적으로....

$$\begin{array}{c} \text{class} \\ \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \end{array} \quad \begin{array}{cc} 1 & 2 \\ \begin{bmatrix} 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{array}$$

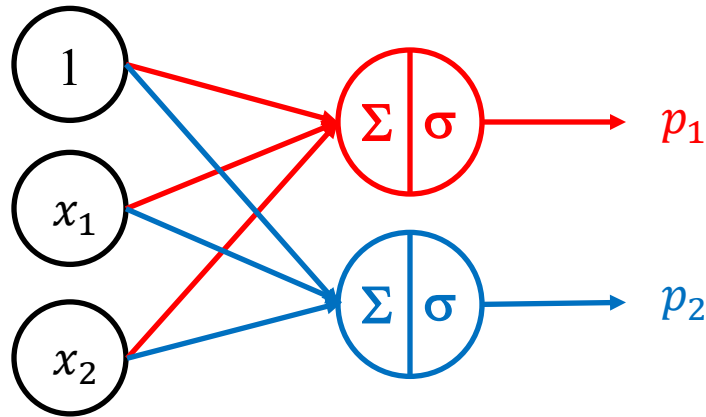
$$\begin{array}{c} \text{class} \\ \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} \end{array} \quad \begin{array}{ccc} 1 & 2 & 3 \\ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \end{array}$$

$$\begin{array}{c} \text{class} \\ \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \end{array} \quad \begin{array}{cccc} 1 & 2 & 3 & 4 \\ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \end{array}$$



One-hot Encoding을 이용한 이진 분류 퍼셉트론

- 두 개의 로지스틱 모델을 사용하여 이진 분류를 수행
 - p_1 과 p_2 를 비교하여 클래스를 결정
 - 중요하게, p_1 과 p_2 는 합이 1이 아님 ($p_1 + p_2 \neq 1$)
 - $L(\mathbf{w}, b) = -\sum_i [y_i \log(p_1) + (1 - y_i) \log(p_2)]$ 는 사용 불가



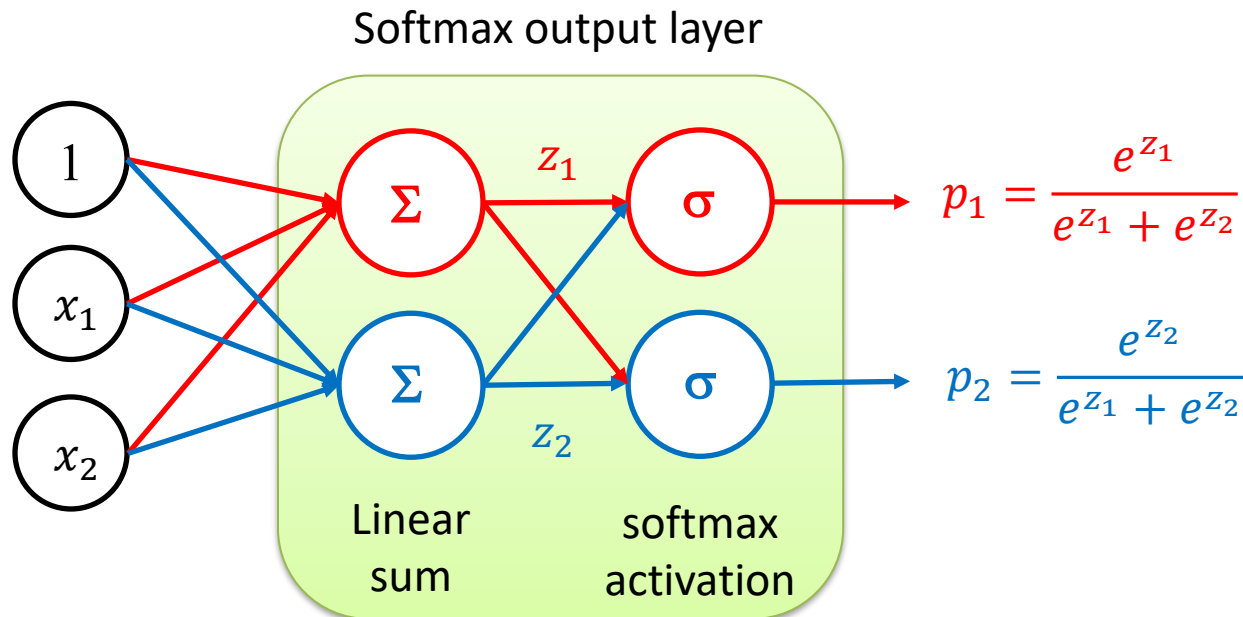
- 레이블링 문제 (Labeling Problem)
 - 샘플이 동시에 여러 개의 클래스에 속할 수 있음
 - 실제로 훈련은 $[1,0]$ 과 $[0,1]$ 로 수행되더라도, $[1,1]$ 이나 $[0,0]$ 을 예측하는 것이 가능
 - 다음과 같은 경우를 고려: $[p_1, p_2]$ is $[0.8, 0.9]$ or $[0.3, 0.1]$
 - 호랑이 vs. 사자 로 훈련하더라도, 라이거나 말에 대한 예측이 가능



소프트맥스 회귀 (Softmax Regression)

- 소프트맥스 회귀 (**softmax regression**) 모델은 시그모이드 대신 소프트맥스 활성화 함수를 사용
 - Softmax activation: $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$
 - 크로스 엔트로피 손실의 정의가 가능

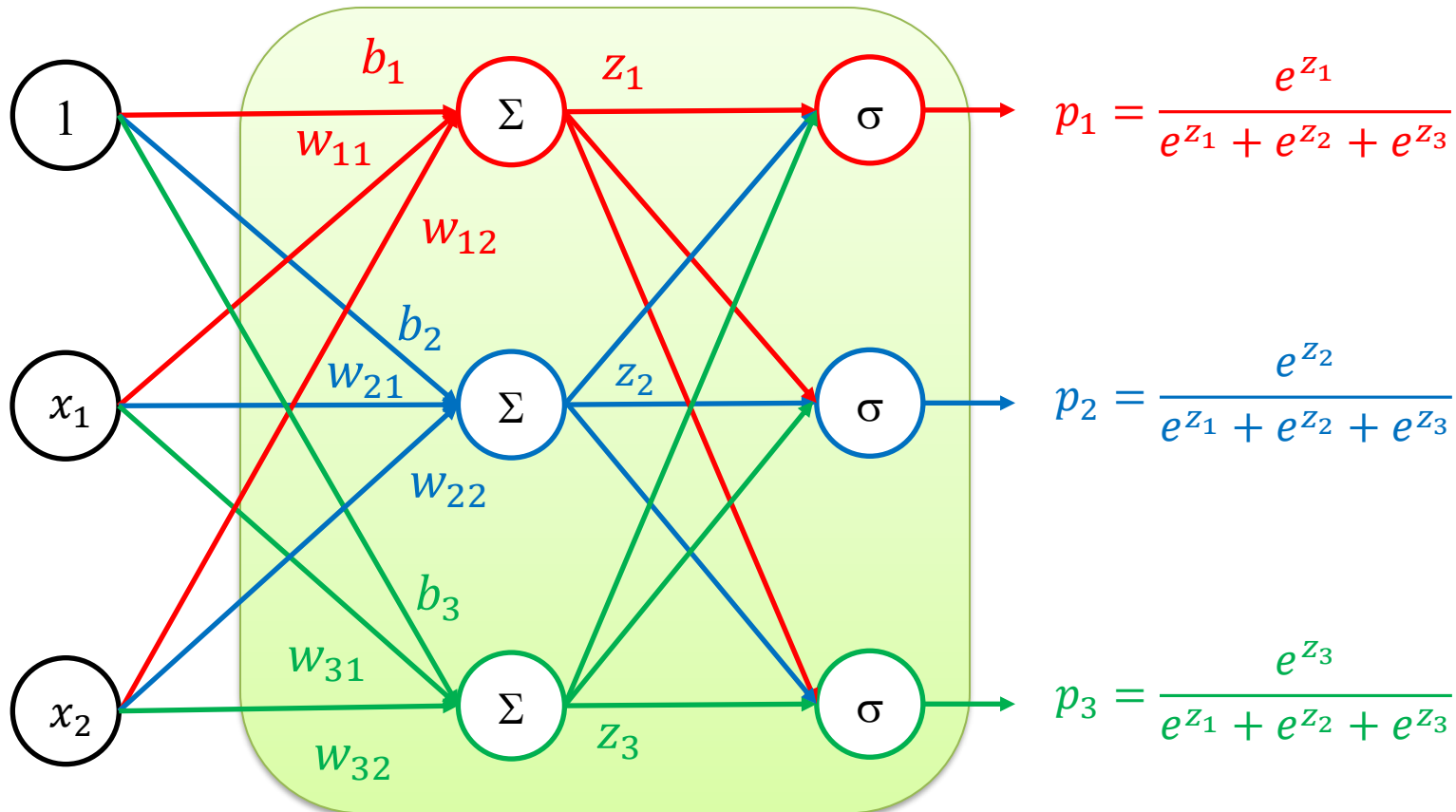
$$L = y_1 \log p_1 + y_2 \log p_2 \text{ where } y_1 + y_2 = 1 \text{ and } p_1 + p_2 = 1.$$



소프트맥스 회귀 (Softmax Regression)

- 소프트맥스는 멀티클래스 분류 문제에 쉽게 적용 가능.
 - 일반적인 크로스 엔트로피 손실 함수 사용

$$L(\mathbf{W}, \mathbf{b}) = -\sum_{k=1}^K y_k \log p_k = -y_1 \log p_1 - y_2 \log p_2 - y_3 \log p_3$$





파라미터 업데이트

- Similarly using gradient descent, $w^{(t+1)} = w^{(t)} - \lambda \frac{\partial L}{\partial w}$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial w_{21}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial w_{21}} + \frac{\partial L}{\partial p_3} \frac{\partial p_3}{\partial w_{21}} = -\frac{y_1}{p_1} \frac{\partial p_1}{\partial w_{21}} - \frac{y_2}{p_2} \frac{\partial p_2}{\partial w_{21}} - \frac{y_3}{p_3} \frac{\partial p_3}{\partial w_{21}}$$

$$\frac{\partial p_1}{\partial w_{21}} = \frac{\partial p_1}{\partial z_2} \frac{\partial z_2}{\partial w_{21}} = p_1(1 - p_1) \frac{\partial z_2}{\partial w_{21}}$$

$$\frac{\partial p_2}{\partial w_{21}} = \frac{\partial p_2}{\partial z_2} \frac{\partial z_2}{\partial w_{21}} = -p_2 p_1 \frac{\partial z_2}{\partial w_{21}}$$

$$\frac{\partial p_3}{\partial w_{21}} = \frac{\partial p_3}{\partial z_2} \frac{\partial z_2}{\partial w_{21}} = -p_3 p_1 \frac{\partial z_2}{\partial w_{21}}$$

$$\frac{\partial z_2}{\partial w_{21}} = x_1$$

- Bring all together... $\frac{\partial L}{\partial w_{21}} = (p_1 - y_1)x_1$

- Generally, for weight to output k from input m , $\frac{\partial L}{\partial w_{km}} = (p_k - y_k)x_m$



파라미터 업데이트

- Using matrix notation ... (keep in mind that it is for one sample)

$$\begin{bmatrix} w_{11}^{(t+1)} & w_{12}^{(t+1)} \\ w_{21}^{(t+1)} & w_{22}^{(t+1)} \\ w_{31}^{(t+1)} & w_{32}^{(t+1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(t)} & w_{12}^{(t)} \\ w_{21}^{(t)} & w_{22}^{(t)} \\ w_{31}^{(k)} & w_{32}^{(k)} \end{bmatrix} - \lambda \begin{bmatrix} \partial L / \partial w_{11} & \partial L / \partial w_{12} \\ \partial L / \partial w_{21} & \partial L / \partial w_{22} \\ \partial L / \partial w_{31} & \partial L / \partial w_{32} \end{bmatrix}$$

$$= \begin{bmatrix} w_{11}^{(t)} & w_{12}^{(t)} \\ w_{21}^{(t)} & w_{22}^{(t)} \\ w_{31}^{(t)} & w_{32}^{(t)} \end{bmatrix} - \lambda \begin{bmatrix} (p_1 - y_1)x_1 & (p_1 - y_1)x_2 \\ (p_2 - y_2)x_1 & (p_2 - y_2)x_2 \\ (p_3 - y_3)x_1 & (p_3 - y_3)x_2 \end{bmatrix}$$

- More simply ...

Jacobian Matrix
= Gradient

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \lambda \nabla_{\mathbf{W}} \mathbf{L}$$

$$\nabla_{\mathbf{W}} \mathbf{L} = (\mathbf{P} - \mathbf{Y})^T \mathbf{X}$$

where

$$\begin{aligned} \mathbf{W} &\in \mathbb{R}^{k \times m} \\ \mathbf{X} &\in \mathbb{R}^{n \times m} \\ \mathbf{Y} &\in \mathbb{R}^{n \times k} \\ \mathbf{P} &\in \mathbb{R}^{n \times k} \end{aligned}$$



파라미터 업데이트

- More closely, *using all samples...*

$$\nabla_{\mathbf{W}} L = (\mathbf{P} - \mathbf{Y})^T \mathbf{X}$$

$$\begin{aligned} \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \cdots & \frac{\partial L}{\partial w_{k1}} \\ \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial w_{1m}} & \cdots & \frac{\partial L}{\partial w_{km}} \end{bmatrix} &= \left(\begin{bmatrix} p_{1,1} & \cdots & p_{1,k} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,k} \end{bmatrix} - \begin{bmatrix} y_{1,1} & \cdots & y_{1,k} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,k} \end{bmatrix} \right)^T \begin{bmatrix} x_{1,1} & \cdots & x_{1,m} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,m} \end{bmatrix} \\ &= \begin{bmatrix} p_{1,1} - y_{1,1} & \cdots & p_{n,1} - y_{n,1} \\ \vdots & \ddots & \vdots \\ p_{1,k} - y_{1,k} & \cdots & p_{n,k} - y_{n,k} \end{bmatrix} \begin{bmatrix} x_{1,1} & \cdots & x_{1,m} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,m} \end{bmatrix} \\ &= \begin{bmatrix} \sum_{i=1}^n (p_{i,1} - y_{i,1}) x_{i,1} & \cdots & \sum_{i=1}^n (p_{i,1} - y_{i,1}) x_{i,m} \\ \vdots & \ddots & \vdots \\ \sum_{i=1}^n (p_{i,k} - y_{i,k}) x_{i,1} & \cdots & \sum_{i=1}^n (p_{i,k} - y_{i,k}) x_{i,m} \end{bmatrix} \end{aligned}$$

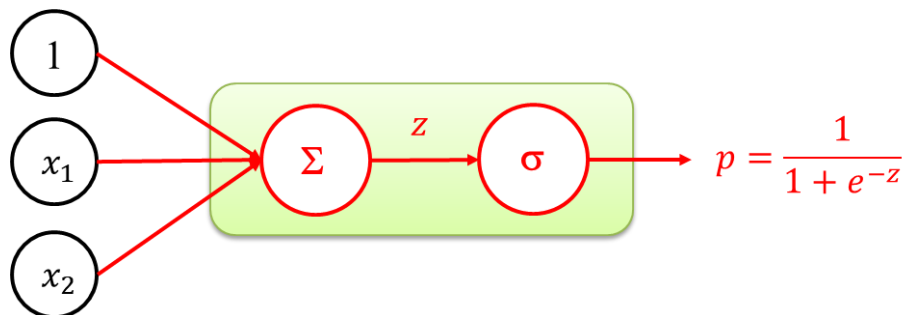


다양한 퍼셉트론의 비교

- 기본적으로 전부 동일하지만 서로 다른 손실 함수를 사용

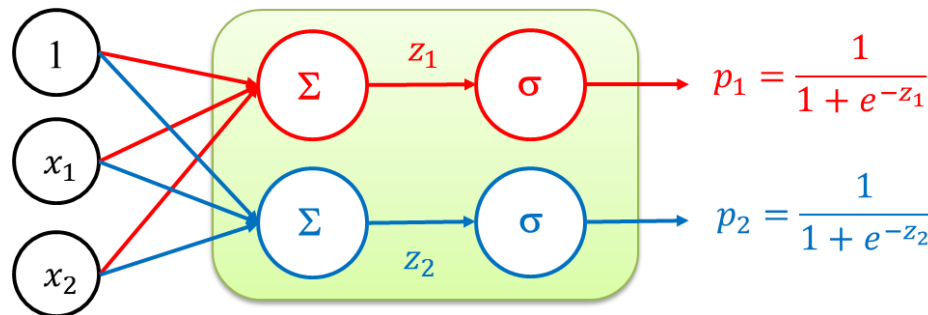
logistic regression
for classification

$$L = -(y \log p + (1 - y) \log(1 - p))$$



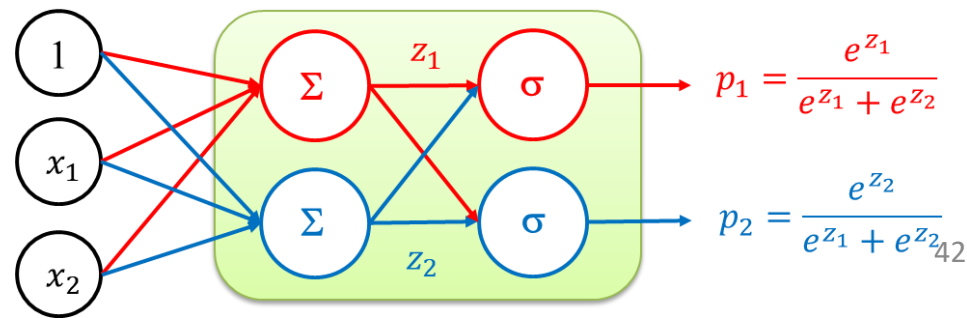
logistic regression with one-hot
(maybe for labeling)

$$L = -(y_1 \log p_1 + (1 - y_1) \log(1 - p_1)) \\ -(y_2 \log p_2 + (1 - y_2) \log(1 - p_2))$$



softmax regression with one-hot
for classification

$$L = -(y_1 \log p_1 + y_2 \log p_2)$$

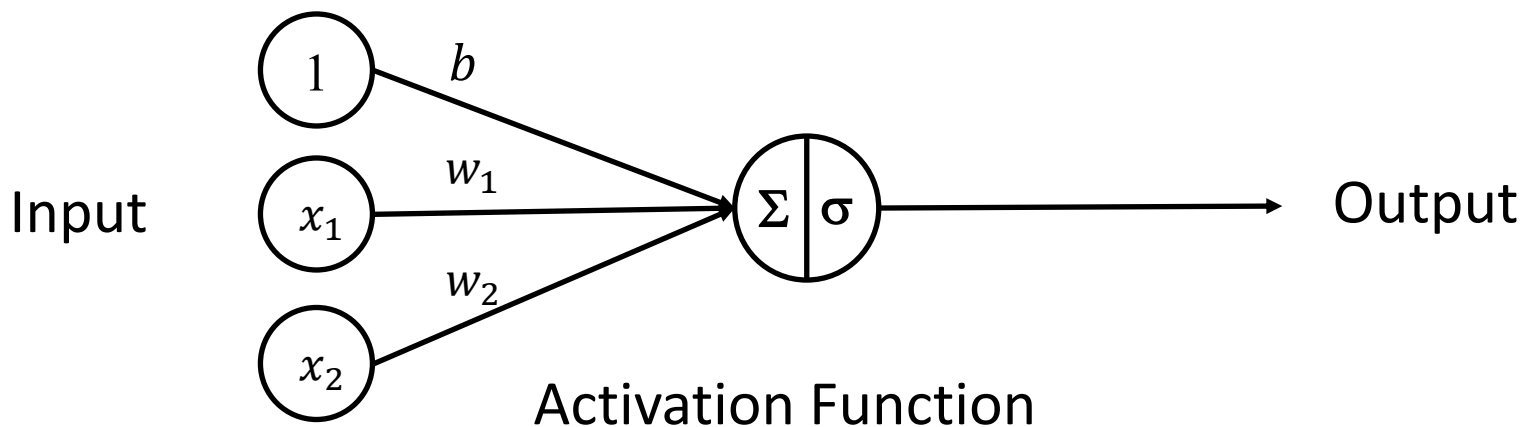


딥러닝 기초

다계층 퍼셉트론

퍼셉트론

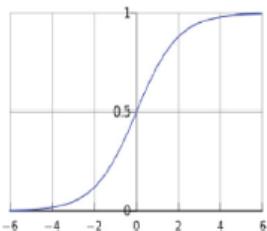
- 뉴런에 대한 수학적 모델: 선형 결합 + 비선형 활성화 함수



$$y = \sigma \left(\sum_j w_j x_j + b \right) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

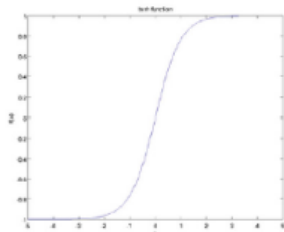
logistic ("sigmoid")

$$f(z) = \frac{1}{1 + \exp(-z)}$$



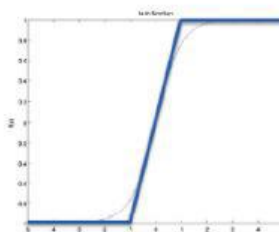
tanh

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



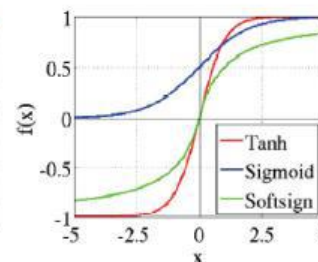
hard tanh

$$\text{HardTanh}(x) = \begin{cases} -1 & \text{if } x < -1 \\ x & \text{if } -1 \leq x \leq 1 \\ 1 & \text{if } x > 1 \end{cases}$$



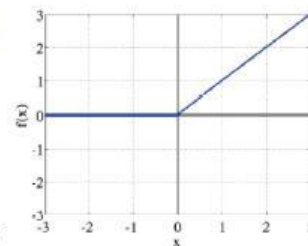
soft sign

$$\text{softsign}(z) = \frac{a}{1 + |a|}$$



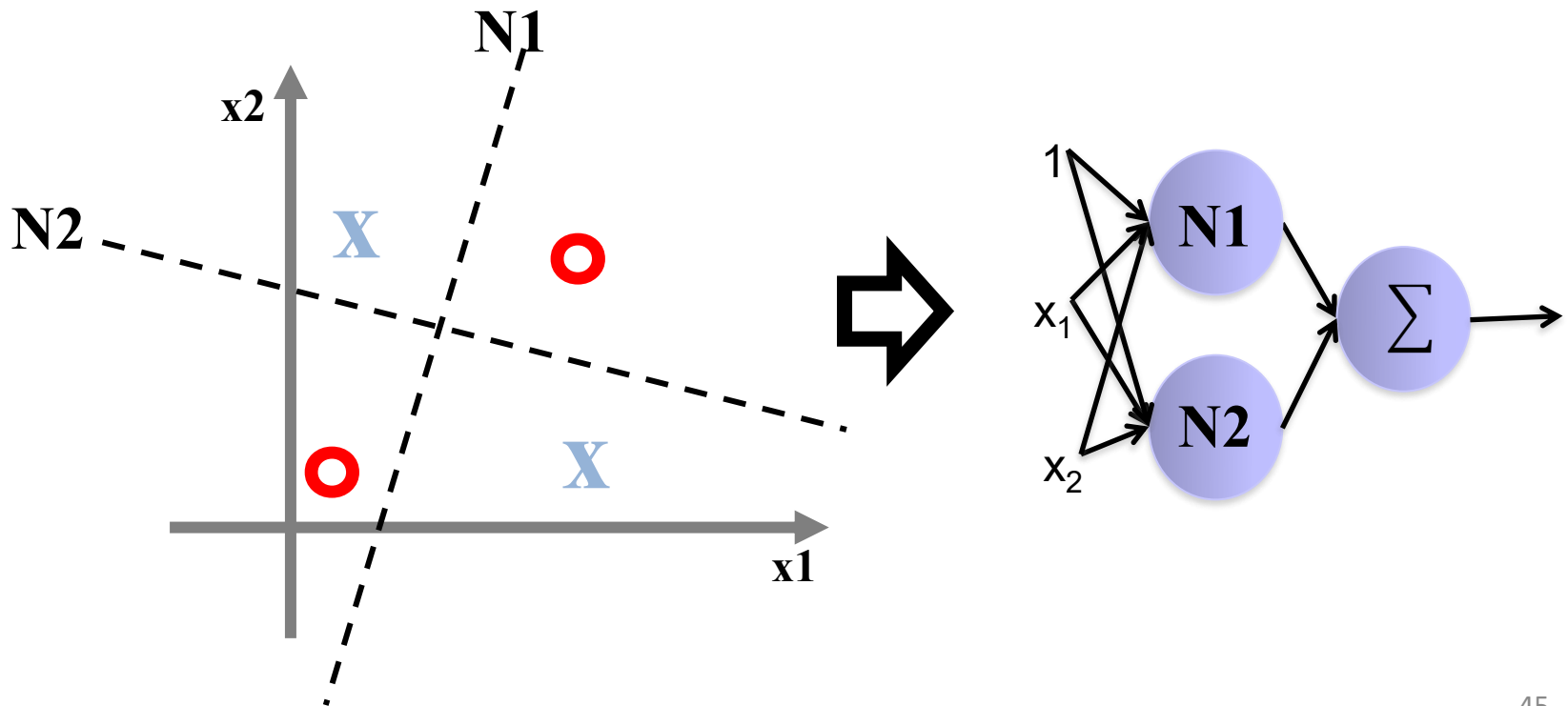
rectifier

$$\text{rect}(z) = \max(z, 0)$$



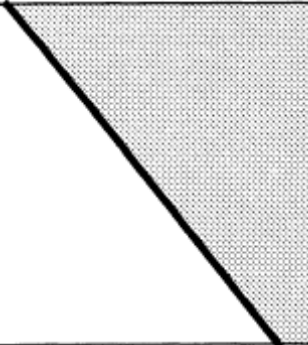
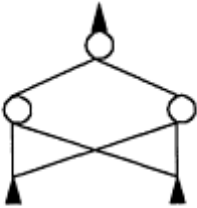
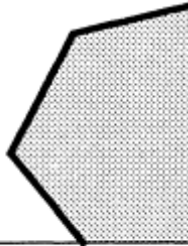
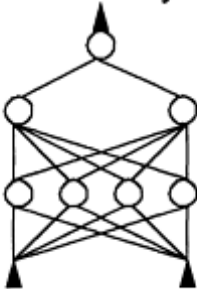
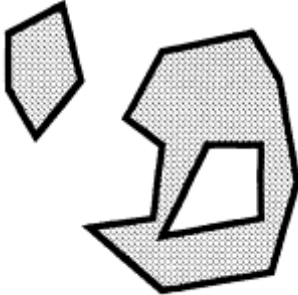
퍼셉트론의 비선형화

- 커널 퍼셉트론
 - 입력에 고차항을 추가 혹은 커널을 이용해 입력 공간을 변형
- 다계층 퍼셉트론 (MLP: Multilayer Perceptron)
 - 퍼셉트론을 쌓아 비선형성을 추가





심층 구조 (Deep Structure)

Structure	Types of Decision Regions	Most General Region Shapes
Single-Layer 	Half-Plane Bounded by A Hyper-Plane	
Two-Layer 	Convex Open, or Closed Regions	
Three-Layer 	Arbitrary (Complexity Limited by the Number of Nodes)	

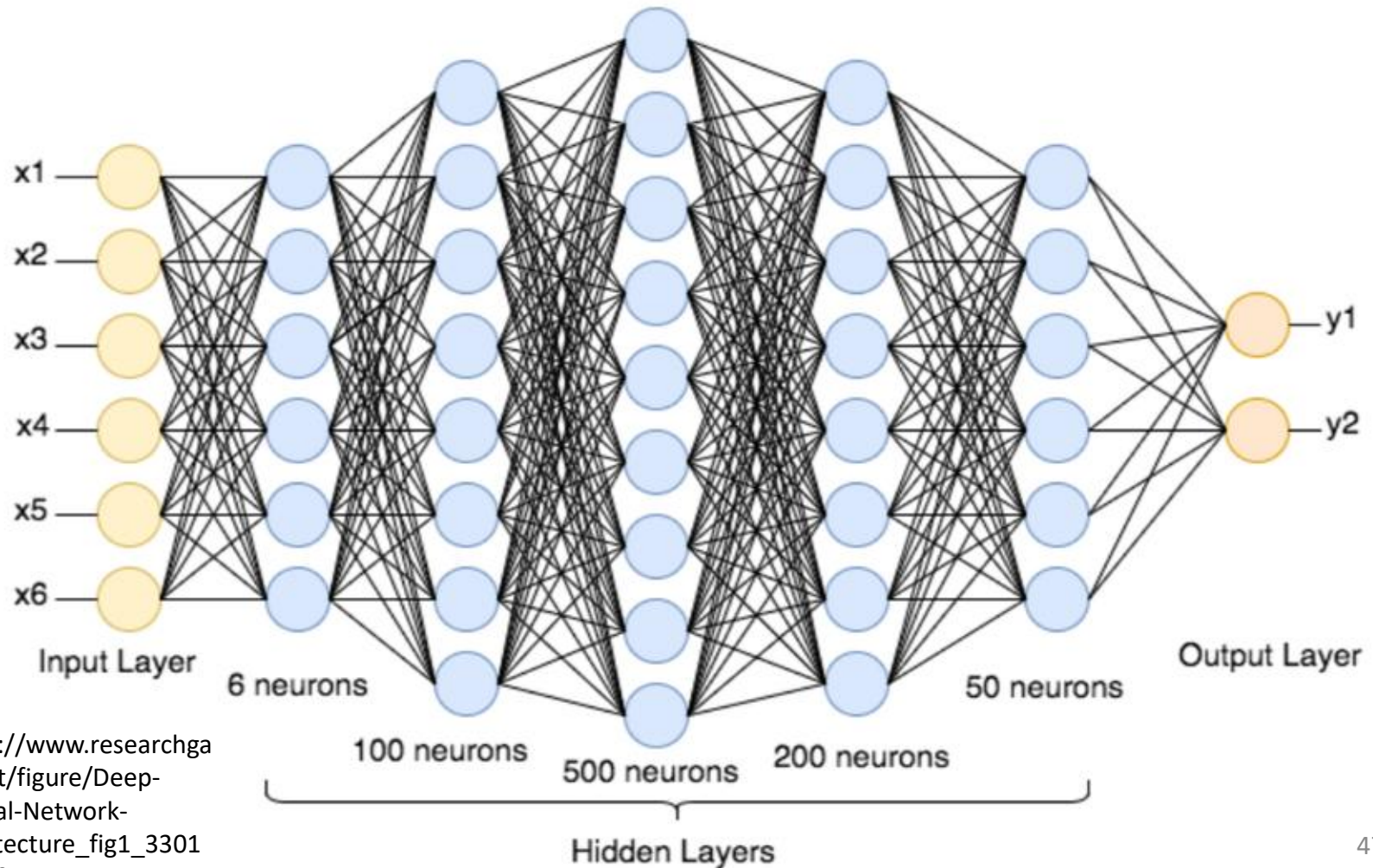
- 더 많이 쌓을 수록 더 복잡한 판단의 경계를 형성

**More Layers
=
Deeper**



심층 신경망 (DNN: Deep Neural Network)

- 심층 신경망: 많은 잠재계층(2개 이상)을 갖는 신경망 구조
 - Shallow network: 0 혹은 1개의 잠재계층을 갖는 구조

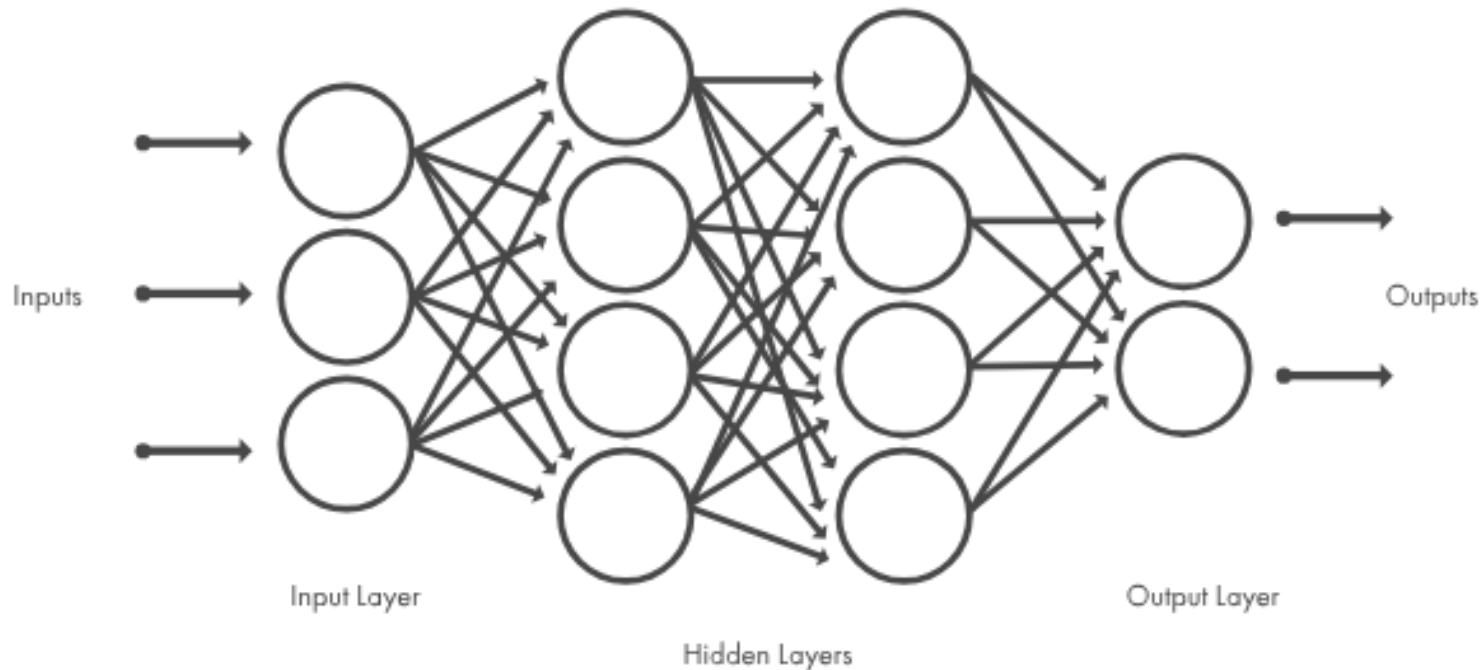


https://www.researchgate.net/figure/Deep-Neural-Network-architecture_fig1_330120030



딥러닝 (Deep Learning)

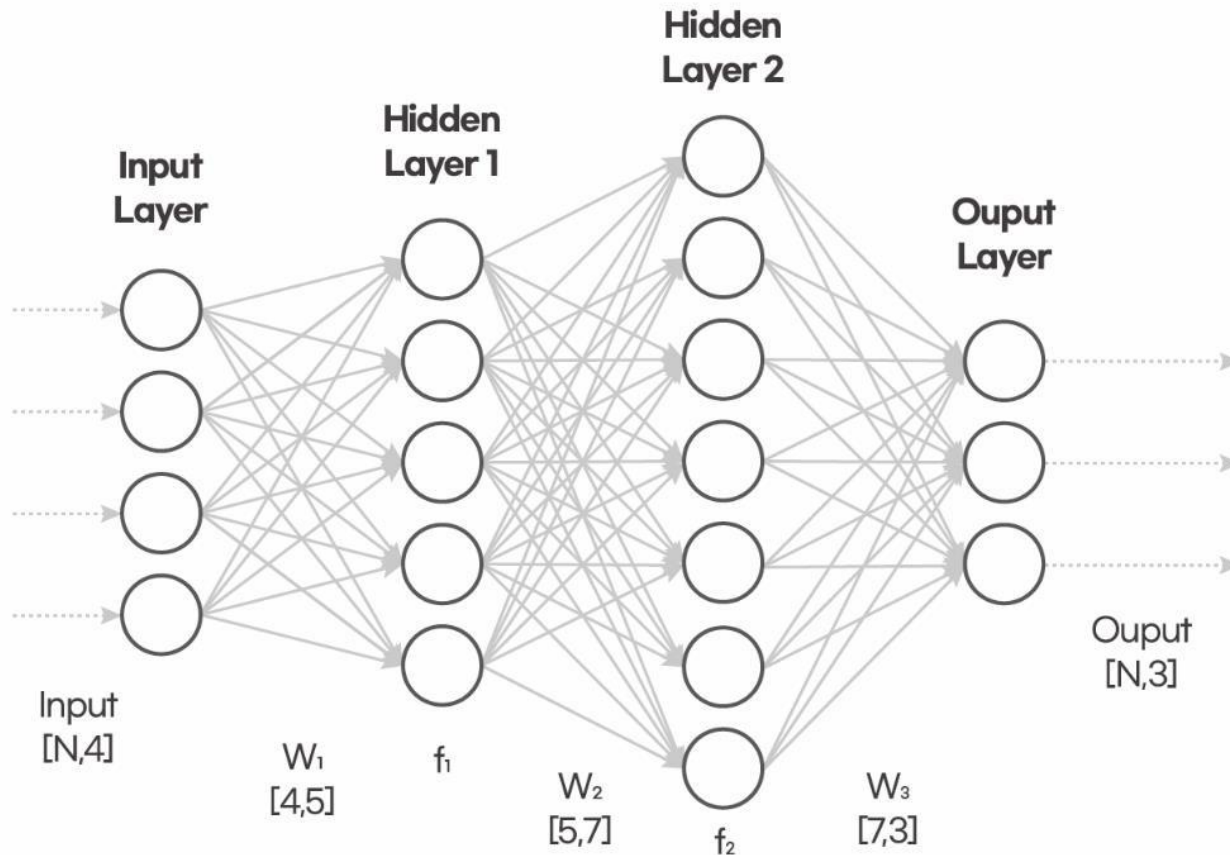
- 딥러닝
 - 외부로 나타나지 않고 숨겨진 계층(hidden layer)가 다수 존재하는 신경망 모델
 - 숨겨진 변수가 학습을 통해 필요한 정보(feature)를 추출하는 모델
- 전통적 접근 vs. 딥러닝 접근
 - 전통적 접근: 인간 전문가가 유용한 정보를 선정하고 이를 데이터에서 계산
 - 딥러닝 접근: 모델이 자동으로 유용한 정보를 추출하여 사용





다계층 퍼셉트론 (MLP: Multi-layer Perceptron)

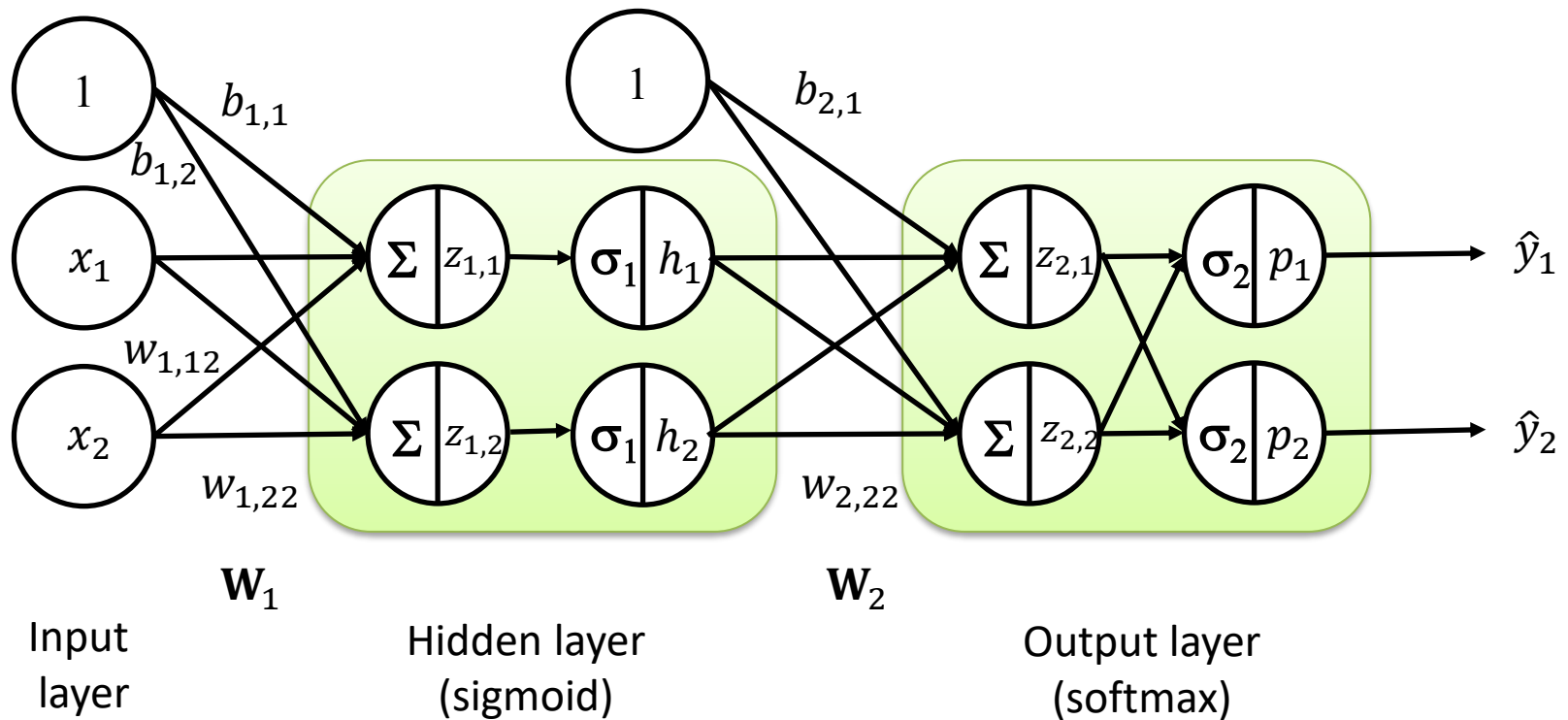
- 다계층 퍼셉트론 (MLP): 가장 기본적인 형태의 인공신경망
 - 모델 파라미터: weights
 - 튜닝 파라미터: # of layer, # of nodes, activation, regularization
 - 학습 파라미터: learning rate, batch size, and etc...





다계층 퍼셉트론 (MLP: Multi-layer Perceptron)

- 예제: sigmoid layer (2) + softmax layer (2), 이진 분류를 위한 크로스 엔트로피 손실 함수 사용
 - 12개의 파라미터가 존재:

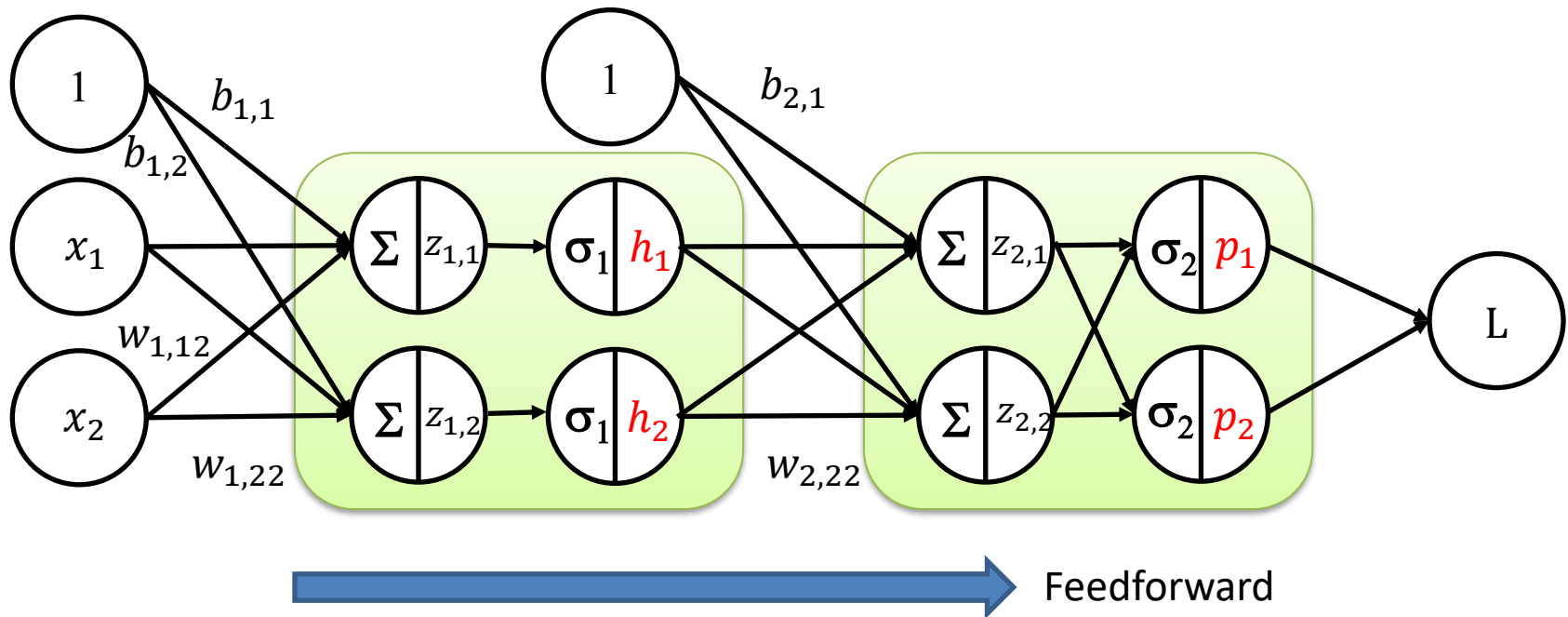


$$L = y_1 \log p_1 + y_2 \log p_2$$



순방향 계산 (Feedforward Computation)

- 순방향 계산: 입력으로부터 파라미터를 통해 출력과 손실을 계산



$$h_1 = \sigma_1(w_{1,11}x_1 + w_{1,12}x_2 + b_{1,1})$$

$$p_1 = \sigma_2(w_{2,11}h_1 + w_{2,12}h_2 + b_{2,1})$$

$$h_2 = \sigma_1(w_{1,21}x_1 + w_{1,22}x_2 + b_{1,2})$$

$$p_2 = \sigma_2(w_{2,21}h_1 + w_{2,22}h_2 + b_{2,2})$$

$$L = y_1 \log p_1 + y_2 \log p_2$$



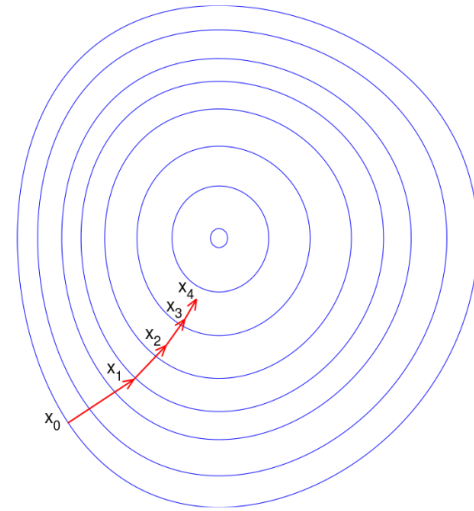
경사하강법을 이용한 손실 함수 최소화

- 손실 함수(loss function)의 최소화
 - 손실함수를 최소화하는 파라미터를 학습
 - 각 파라미터에 대한 손실함수의 미분값(경사도, gradient)를 0으로 하는 값
- 경사하강법 (gradient descent)

$\theta^{(0)}$: random position

$$\theta^{(i+1)} = \theta^{(i)} - \lambda \frac{\partial L(\theta^{(i)})}{\partial \theta}$$

각 파라미터에 대한
경사도의 계산이 핵심!

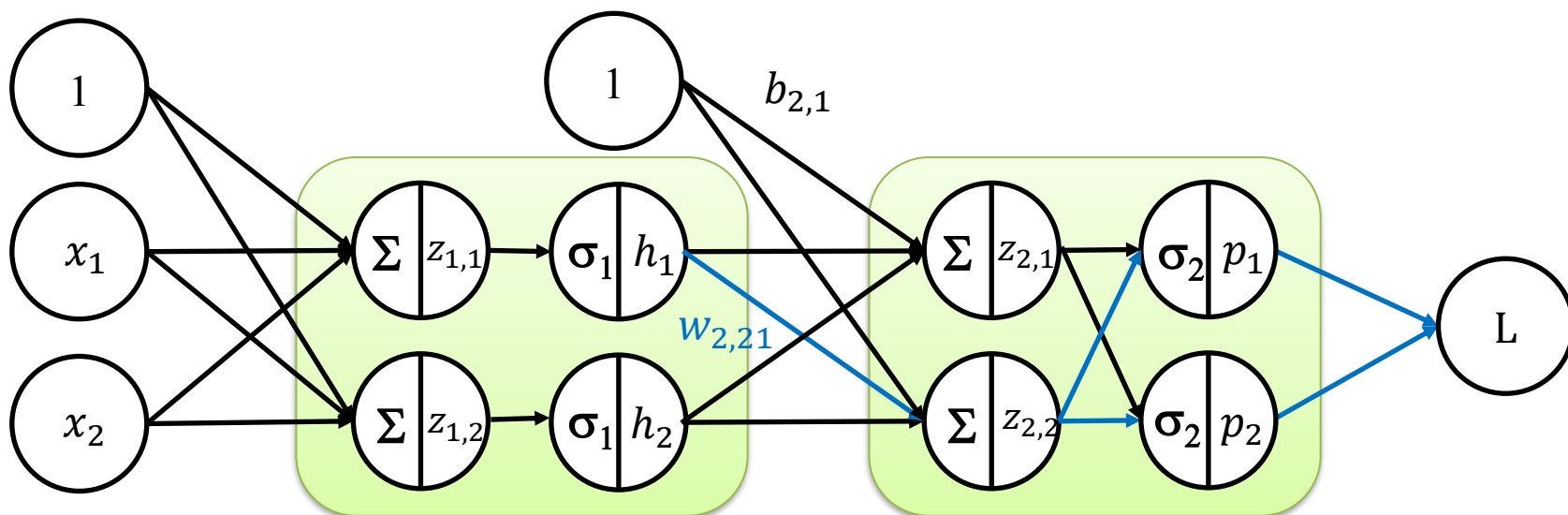


https://en.wikipedia.org/wiki/Gradient_descent



경사도 계산

- $w_{2,21}$ 에 대한 경사도 계산



$$\frac{\partial L}{\partial w_{2,21}} = \frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

$$= \left(\frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \right) \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

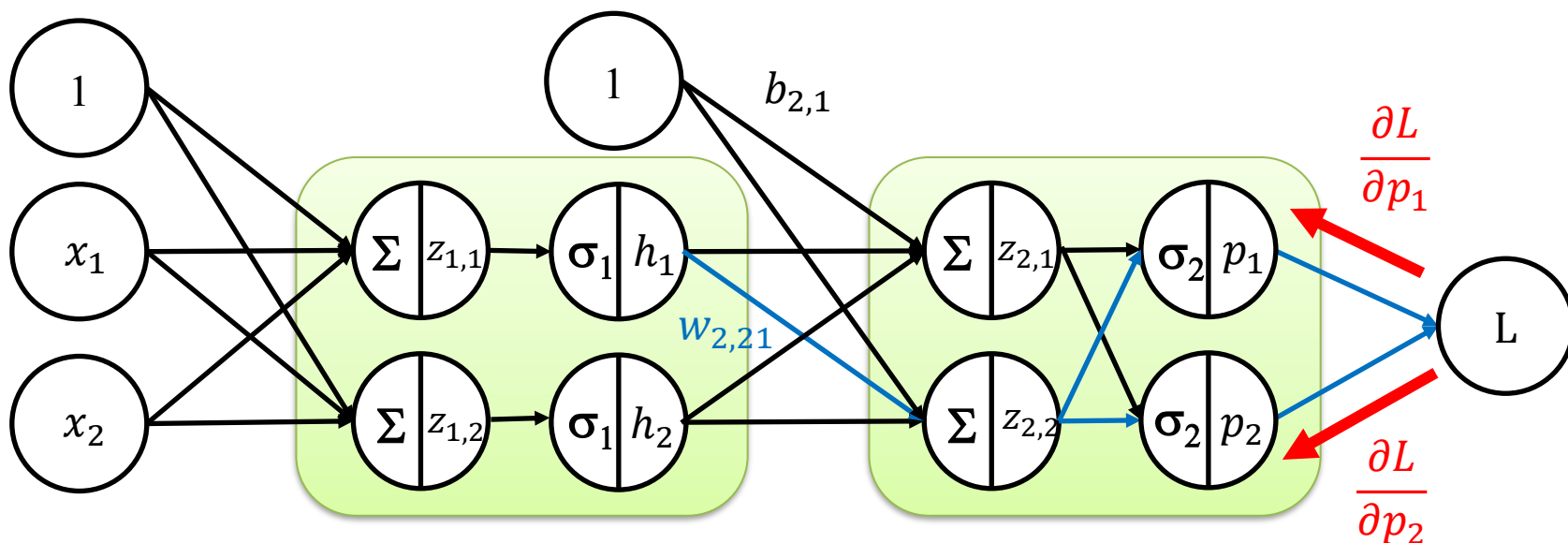
$$= \left(-\frac{y_1}{p_1} \cdot p_1(1-p_1) - \frac{y_2}{p_2} \cdot p_2(1-p_2) \right) \cdot h_1$$

순방향에서 계산된 값들



경사도 계산

- $w_{2,21}$ 에 대한 경사도 계산



$$\frac{\partial L}{\partial w_{2,21}} = \frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

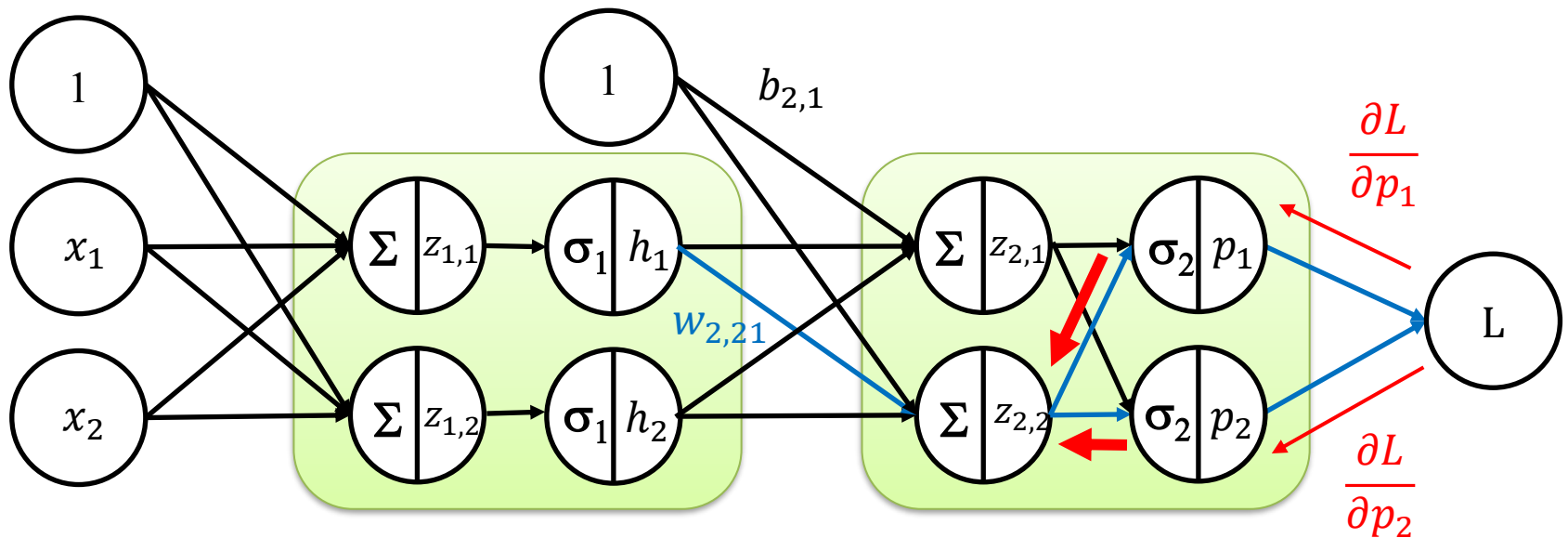
$$= \left(\frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \right) \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

$$= \left(-\frac{y_1}{p_1} \cdot p_1(1-p_1) - \frac{y_2}{p_2} \cdot p_2(1-p_2) \right) \cdot h_1$$

순방향에서 계산된 값들

경사도 계산

- $w_{2,21}$ 에 대한 경사도 계산



$$\frac{\partial L}{\partial w_{2,21}} = \frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

$$= \left(\frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \right) \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

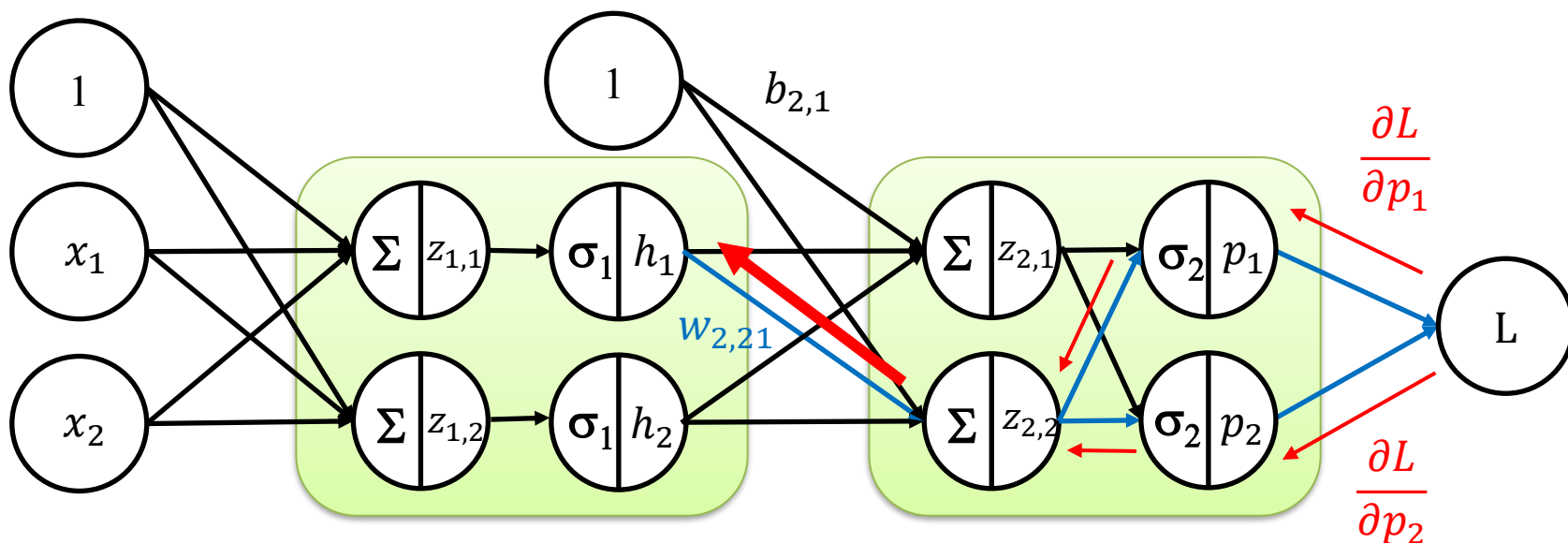
$$= \left(-\frac{y_1}{p_1} \cdot p_1(1-p_1) - \frac{y_2}{p_2} \cdot p_2(1-p_2) \right) \cdot h_1$$

순방향에서 계산된 값들



역방향 경사도 계산

- $w_{2,21}$ 에 대한 경사도 계산



$$\frac{\partial L}{\partial w_{2,21}} = \frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

$$= \left(\frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \right) \frac{\partial z_{2,2}}{\partial w_{2,21}}$$

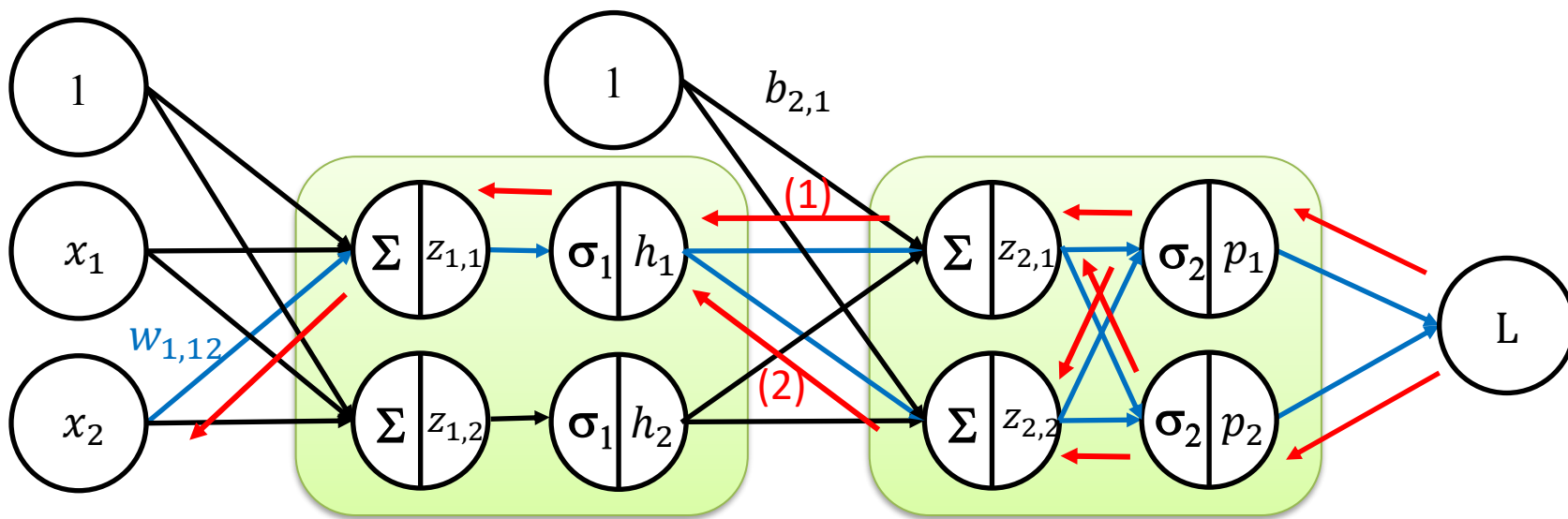
$$= \left(-\frac{y_1}{p_1} \cdot p_1(1 - p_1) - \frac{y_2}{p_2} \cdot p_2(1 - p_2) \right) \cdot h_1$$

순방향에서 계산된 값들



역방향 경사도 계산

- 또 다른 예제, $w_{1,12}$ 에 대한 경사도 계산



$$(1) = \left(\frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,1}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,1}} \right) \frac{\partial z_{2,1}}{\partial h_1}$$

$$(2) = \left(\frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial z_{2,2}} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial z_{2,2}} \right) \frac{\partial z_{2,2}}{\partial h_1}$$

$$\frac{\partial L}{\partial w_{1,12}} = [(1) + (2)] \frac{\partial h_1}{\partial z_{1,1}} \frac{\partial z_{1,1}}{\partial w_{1,12}}$$



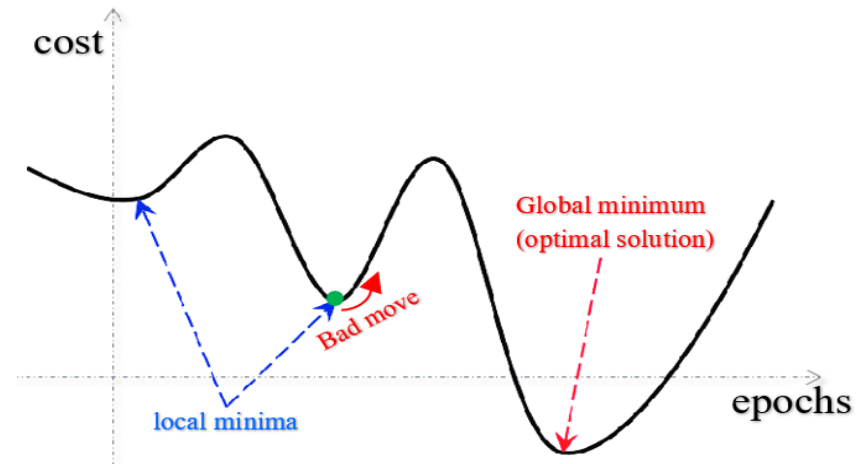
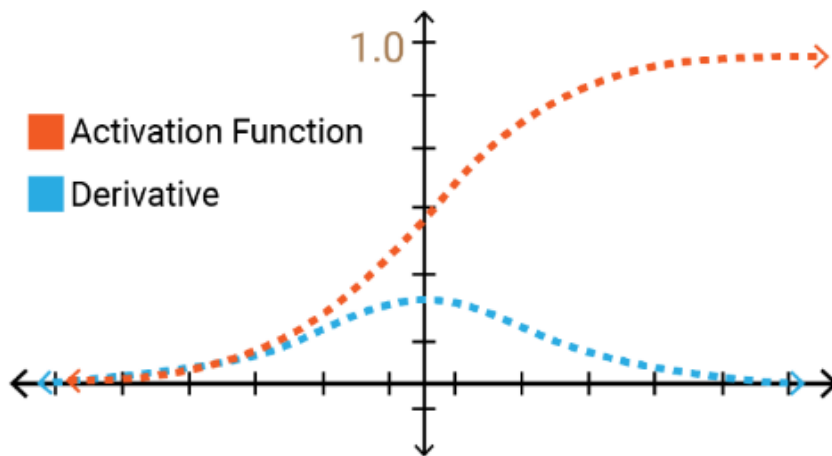
역전파 (Backpropagation)

- **역전파 알고리즘**은 신경망의 가중치를 학습하는 주요한 방법으로 순방향과 역방향 계산을 반복함으로써 수행됨
 - 오늘날 신경망 훈련에 가장 많이 사용되는 알고리즘
- **대략적 알고리즘**
 - (1) 가중치의 초기화 (보통 랜덤한 값으로)
 - (2) 순방향 계산을 통해 출력과 손실을 계산
 - (3) 역방향 계산을 통해 경사치를 계산하고 파라미터를 업데이트
 - (4) 수렴할 때까지 (2)와 (3)을 반복
- **역전파 알고리즘**은 생물학적 지식과 관련이 없음
 - 신경망은 생물학적 신경세포로부터 발전했지만, 생물학적으로 역전파와 같은 원리는 존재하지 않음
 - 역전파는 많은 양의 데이터가 필요한데 비해 생물학적 신경세포는 더 적은 데이터로부터 좀 더 천천히 학습함
 - 새로운 학습 방법이 존재할 가능성!!



심층 신경망 학습의 문제

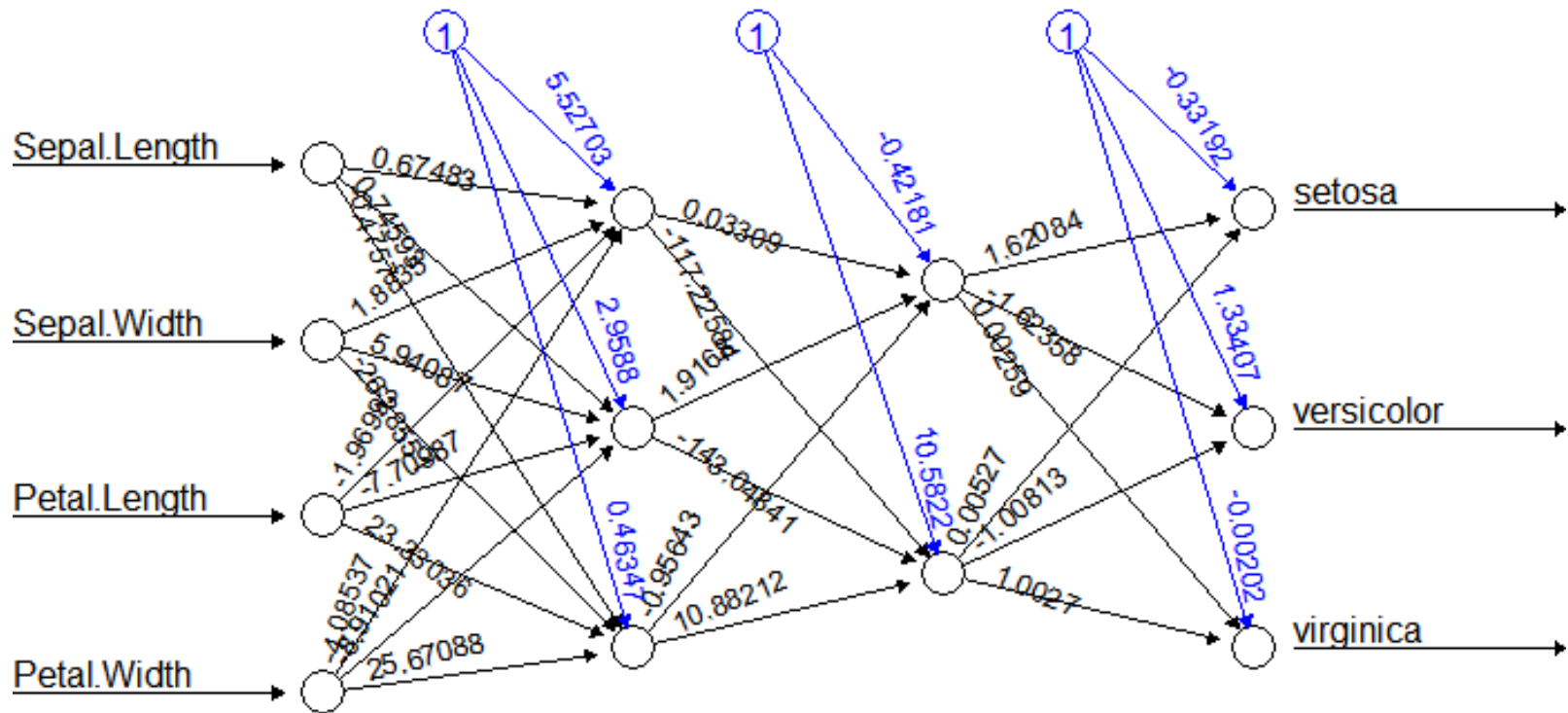
- **경사도 소멸 (vanishing gradient):** 역전파에 따라 경사도가 점점 작아지고 결국 소멸하는 현상
 - 보통 활성화 함수의 미분값이 1보다 작기 때문에 발생하는 현상
 - 경사도 소멸은 심층 신경망의 훈련을 어렵게 하는 주된 요인
 - 이를 방지하기 위한 다양한 기법 (e.g. ReLU, LSTM, ResNet)이 고안됨
- **국소 최적화 (local minimum):** 손실함수가 충분히 최적화 되지 않았음에도 경사도가 0이 되는 문제
 - 경사하강법 류의 방법에서 공통적으로 발생하는 문제
 - 완전히 해결하는 것은 불가능하기 때문에 좋은 시작점을 반복적으로 찾고자 함





예제: 아이리스 데이터셋

- 아이리스 데이터 셋: 4개의 입력에 대하여 3개의 클래스를 분류하는 문제
 - Network architecture: 4 – (3, 2) – 3
 - Sigmoid activation function



Error: 0.001415 Steps: 2062

딥러닝 기초

다양한 학습 기법



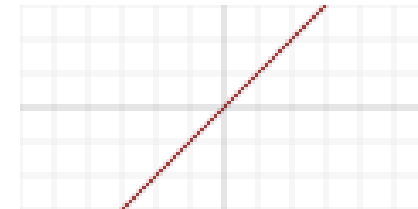
활성화 함수

- **Identity**

- 선형 활성화 함수

$$\sigma(z) = z$$

$$\sigma'(z) = 1$$

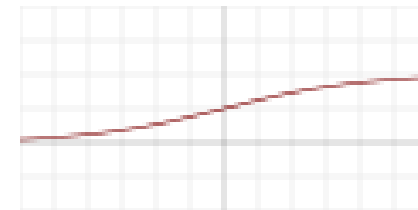


- **Sigmoid (Logistic)**

- 기본적인 활성화 함수
- 경사도 소멸의 문제가 발생
- 항상 양수만을 도출하여 편향이 발생

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

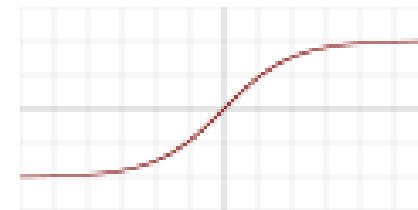


- **Hyperbolic tangent**

- Sigmoid와 유사하나 0 중심
- 양수/음수 모두 도출되어 편향이 없음
- 경사도 소멸의 문제가 상대적으로 적지만 여전히 존재

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\sigma'(z) = 1 - \sigma(z)^2$$



https://en.wikipedia.org/wiki/Activation_function

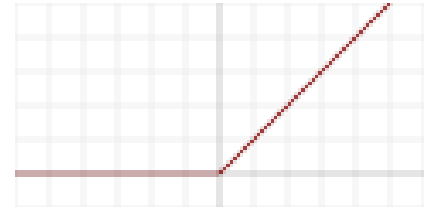


활성화 함수

- **ReLU (Rectifier Linear Unit)**

- 일반적으로 심층신경망에 가장 많이 사용되는 함수
- 장점: 희소 표현이 가능. 경사도 소멸이 적음, 계산이 효율적
- 단점: 양수만이 도출, 값의 제한이 없음, 모든 경사도가 0이 되는 현상이 발생

$$\sigma(z) = \max(0, z)$$
$$\sigma'(z) = 0 \text{ or } 1$$

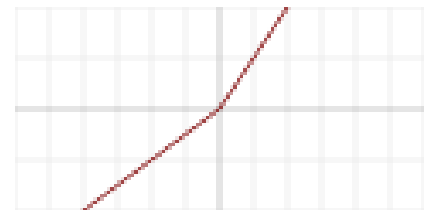


- **Leaky ReLU**

- ReLU을 보완하기 위한 변형
- 경사도가 0이 되는 현상을 방지
- GELU, ELU, SELU 등의 다양한 변형

$$\sigma(z) = \max(\alpha z, z)$$
$$\sigma'(z) = \alpha \text{ or } 1$$

보통 $\alpha = 0.01$



https://en.wikipedia.org/wiki/Activation_function



손실함수 최소화 기법

- 손실함수 최소화
 - 딥러닝 모델 훈련의 핵심
 - 손실함수를 최소화하는 파라미터를 찾고자 함
- 경사하강법 (gradient descent)
 - 임의의 파라미터에서 시작하여 경사를 따라 파라미터의 변화가 0이 될 때까지 업데이트
 - 배치 경사하강법: 모든 데이터를 이용
 - 확률적 경사하강법: 하나의 샘플을 이용
 - 미니배치 경사하강법: 소수의 샘플을 이용
- 경사하강법의 문제
 - 학습률을 정하기가 어려움
 - 너무 큰 학습률: 학습은 빠르지만 불안정
 - 너무 작은 학습률: 안정적이지만 학습이 느림

$$\min_{\theta} L(\theta)$$

$\theta^{(0)}$: random position

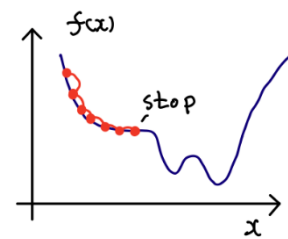
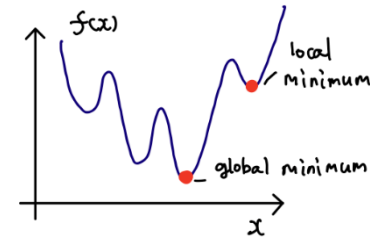
$$\theta^{(i+1)} = \theta^{(i)} - \lambda \nabla L(\theta^{(i)})$$



손실함수 최소화 기법

• 모멘텀 최적화 (Momentum Optimizer)

- 관성 계수를 추가하여 경사값을 업데이트
- 지역최저점이나 안장점(최저점이 아니지만 경사도가 0인 지점)의 문제를 해결
- 관성에 의해 더 빠른 학습이 가능



$$m^{(i)} = \beta_1 m^{(i-1)} + (1 - \beta_1) \nabla L(\theta^{(i)})$$

$$\theta^{(i+1)} = \theta^{(i)} - \lambda m^{(i)}$$

<https://icim.nims.re.kr/post/easyMath/428>

• Adaptive Gradient (Adagrad)

- 일반적인 GD는 모든 파라미터가 같은 학습률로 학습되어 비효율적
- 변화가 적은 파라미터는 더 큰 학습률을 변화가 많이 파라미터를 작은 학습률로 업데이트

k 번째 파라미터 θ_k 에 대한
변화량 제공의 합

$$g_k^{(i)} = g_k^{(i-1)} + \nabla_k L(\theta^{(i)})^2$$

$$\theta_k^{(i+1)} = \theta_k^{(i)} - \frac{\lambda}{\sqrt{g_k^{(i)} + \epsilon}} \nabla_k L(\theta^{(i)})$$



손실함수 최소화 기법

- **RMSProp**

- Adagrad는 점차 학습률이 너무 작아져 더 이상 학습이 진행되지 않는 문제가 발생
- $g_k^{(i)}$ 계산에 이동평균을 적용하여 0으로 빠르게 떨어지는 것을 방지

$$g_k^{(i)} = \beta g_k^{(i-1)} + (1 - \beta) \nabla_k L(\theta^{(i)})^2$$

$$\theta_k^{(i+1)} = \theta_k^{(i)} - \frac{\lambda}{\sqrt{g_k^{(i)} + \epsilon}} \nabla_k L(\theta^{(i)})$$

- **Adam (Adaptive Momentum Estimation)**

- Momentum + RMSProp
- Momentum을 이용한 빠르고 지속적인 업데이트
- RMSProp을 이용한 파라미터 별 차등적인 업데이트

$$m_k^{(i)} = \beta_1 m_k^{(i-1)} + (1 - \beta_1) \nabla_k L(\theta^{(i)})$$

$$g_k^{(i)} = \beta_2 g_k^{(i-1)} + (1 - \beta_2) \nabla_k L(\theta^{(i)})^2$$

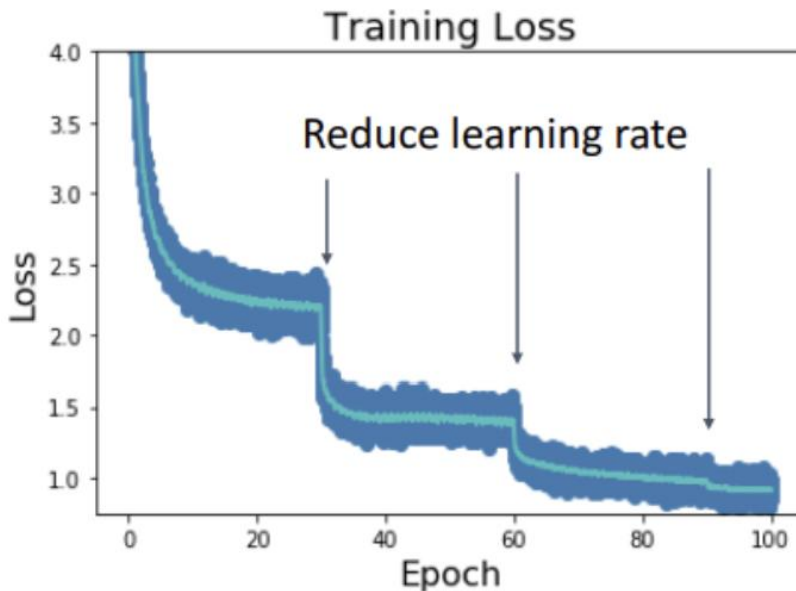
$$\theta_k^{(i+1)} = \theta_k^{(i)} - \frac{\lambda}{\sqrt{g_k^{(i)} + \epsilon}} m_k^{(i)}$$



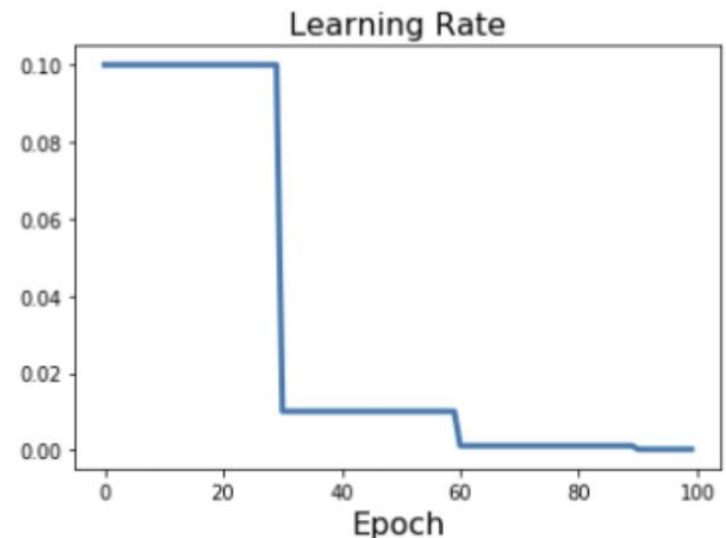
손실함수 최소화 기법

- **Learning Rate Decay**

- 여러가지 방식에도 불구하고 최적화가 진행될 수록 학습률은 작아져야함
- 학습에 따라 학습률을 줄여가는 방식



Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.



<http://cs231n.stanford.edu/>

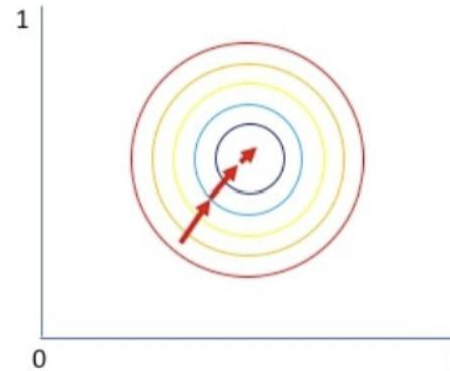
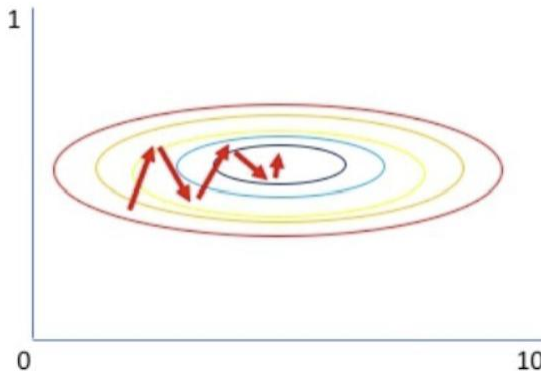


정규화 (Normalization)

- 정규화(Normalization)란?

- 신경망 계층의 입력으로 들어가는 데이터의 분포를 일정하게 맞추어 주는 과정
- 안정적이고 빠른 학습을 위해 주로 사용함

<https://medium.com/@limxiuxian98/batch-normalization-what-should-we-know-8c6f5d30d3ad>



- 데이터 정규화 (Data Normalization) or Data Scaling

- 입력 계층에 들어가는 주어진 데이터(X)를 정규화하는 과정
- 모델 훈련의 일부가 아니라 전처리의 일부
- Min-max normalization: 데이터를 0~1사이로 정규화
- Standardization: 데이터를 평균 0, 분산 1로 정규화



정규화 (Normalization)

- 계층내 정규화 (In-layer Normalization)
 - 신경망 내부에서 한 계층에서 다음 계층으로 전달되는 입력을 정규화
- 배치 정규화 (Batch Normalization)
 - 미니배치 내에서 각 변수가 같은 분포를 갖도록 정규화 (across-sample)
 - 주로 이미지 분석(CNN모델)에 사용
- 계층 정규화 (Layer Normalization)
 - 하나의 표본 내에서 변수들의 같은 분포를 갖도록 정규화 (across-variable)
 - 주로 텍스트 분석(RNN모델)에 사용
- 가중치 정규화 (Weight Normalization)
 - 데이터 대신에 모델의 가중치를 정규화

변수별로
학습되는
파라미터

$$x_n = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

미니배치

	s_1	s_2	s_3
x_1	1	0	5
x_2	2		
x_3	0		
x_4	4		
x_5	3		

$$\mu = 2, \sigma^2 = 7$$

$$\mu = 2, \sigma^2 = 3.5$$



규제화 (Regularization)

- **규제화(Regularization)이란?**
 - 모델의 과대적합(overfitting)을 방지하기 위해 모델의 복잡성을 줄이는 방식
- **Weight Decay**
 - 학습 과정에서 큰 파라미터에 대해서는 큰 페널티를 부여하여 파라미터가 너무 커지는 것을 방지, 손실함수에 벌점항을 추가하여 구현
 - 근본적으로 "모델 선택과 확장"에서 다룬 규제화와 동일
 - 벌점은 L_0 , L_1 , L_2 , $L_{infinite}$ 등이 가능

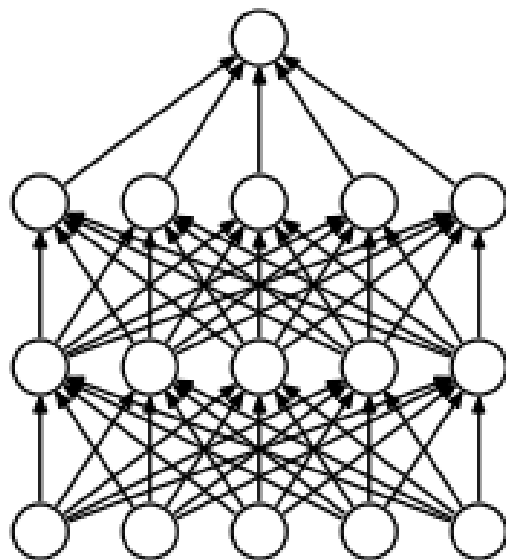
$$\text{Loss}(\theta, \lambda | X, Y) = \text{Error}(\theta | X, Y) + \lambda \text{Penalty}(\theta)$$

$$\text{Loss} = - \sum_i \left[y_i \log \left(\frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}} \right) + (1 - y_i) \log \left(\frac{1}{1 + e^{\beta_0 + \beta_1 x}} \right) \right] + \lambda \beta_1^2$$

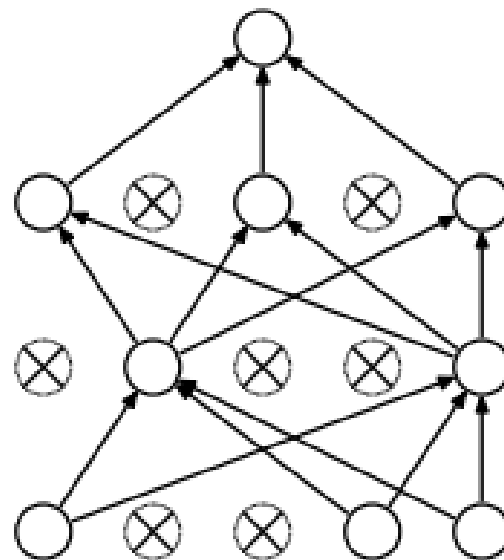
규제화 (Regularization)

- **Dropout**

- 훈련과정에서 노드를 임의로 삭제하여 출력이 일부 노드에만 의존하는 현상을 방지 → 좀 더 안정적인 예측이 가능
- 훈련 과정에서는 매 순방향 계산에서 p 의 비율로 노드를 삭제하고, 그 대신 각 출력을 $1/(1 - p)$ 만큼 스케일하여 계산
- 예측을 할 때는 모든 노드를 그대로 사용



(a) Standard Neural Net



(b) After applying dropout.

딥러닝 기초

텐서플로우



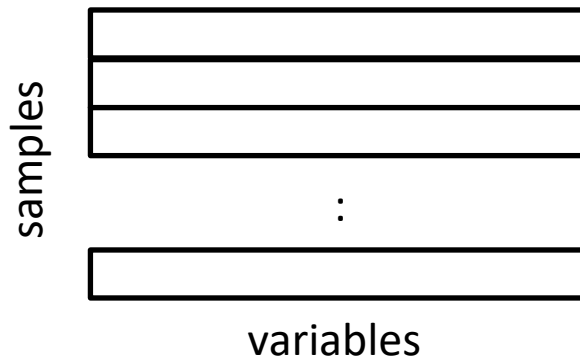
텐서플로우 (Tensorflow)

- 초기의 기계학습 알고리즘은 각자 고유한 훈련 알고리즘과 이를 구현한 소프트웨어 라이브러리로 구성
- 기계학습의 발전과 더불어 많은 알고리즘들이 공통적으로 그래프 모델로 표현될 수 있음이 알려짐
- 기계학습 모델을 공통적으로 표현하고 계산할 수 있는 전용 플랫폼을 개발하기 시작
- **텐서플로우(Tensorflow)**, <https://www.tensorflow.org/>
 - 2015년 구글에서 심층신경망 훈련을 위해 개발되었고, 2019년 2.0이 출시되면서 사용자 편의성이 높아짐
 - 텐서 데이터에 대한 그래프 모델과 다양한 손실함수, 자동화된 미분 기능 등을 제공
 - 모델의 개발, 배포, 상용제품의 개발 및 학습 파이프라인까지 모델 개발 전 과정에 사용될 수 있음
 - 2.0부터 공식적으로 Keras를 사용자 인터페이스로 사용

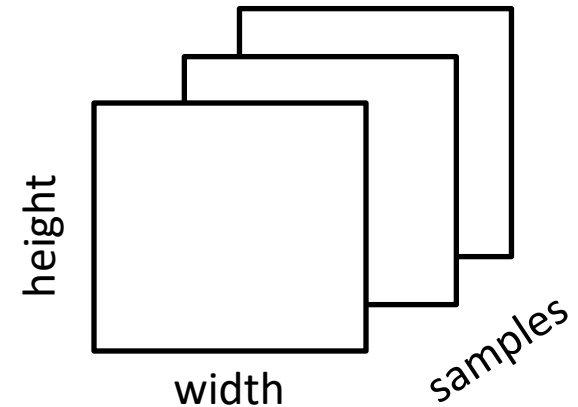


텐서 (Tensor)

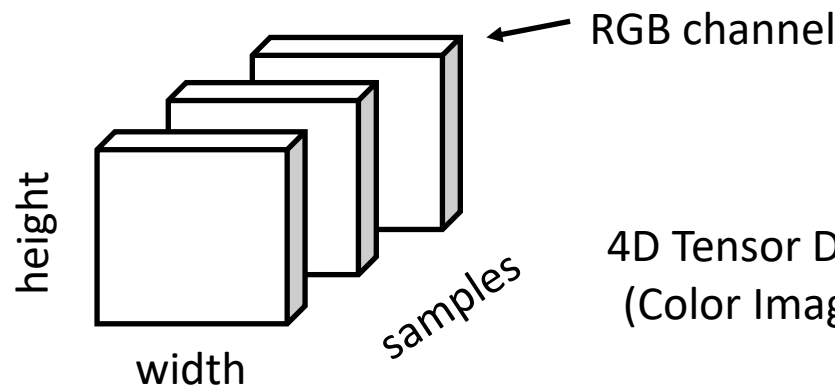
- **텐서**: n차원의 배열
 - 0차원 텐서: 스칼라, 1차원 텐서: 벡터, 2차원 텐서: 행렬
- 기계학습에서 데이터는 텐서로 표현



2D Tensor Data
(Tabular)



3D Tensor Data
(BW Image, Time-course)

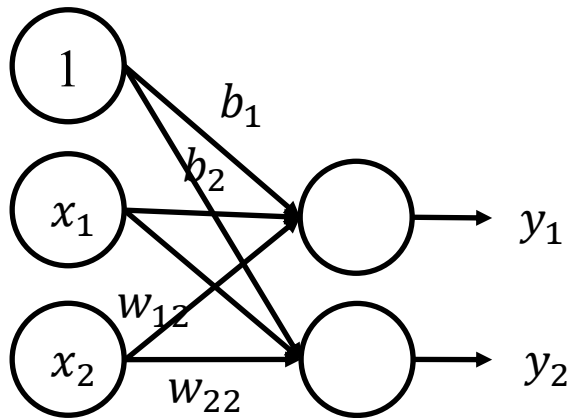


4D Tensor Data
(Color Image)



텐서의 흐름

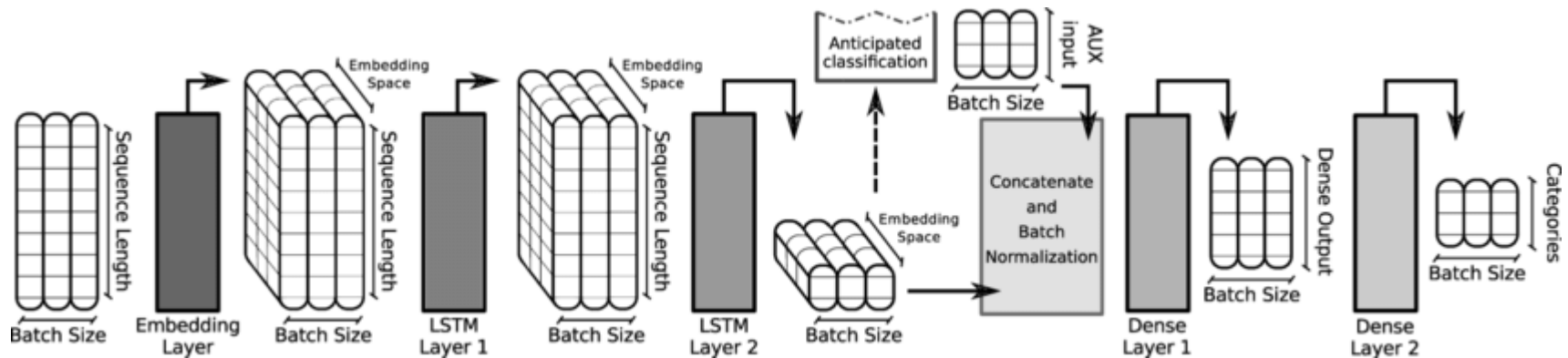
- 신경망의 주된 계산은 텐서의 곱셈과 덧셈
- 계산된 텐서는 다음 레이어로 흘러 들어감
- 병렬화가 쉬워 GPU 연산에 유리



$$\sigma(w_{11}x_{11} + w_{12}x_{12} + b_1) = y_{11}$$

$$\sigma(w_{21}x_{11} + w_{22}x_{12} + b_2) = y_{12}$$

$$\sigma \left(\begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ \vdots & \vdots \\ x_{n1} & x_{n2} \end{bmatrix} \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \\ b_1 & b_2 \\ \vdots & \vdots \\ b_1 & b_2 \end{bmatrix} \right) = \begin{bmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \\ \vdots & \vdots \\ y_{n1} & y_{n2} \end{bmatrix}$$





Tensorflow vs. PyTorch

	Tensorflow	Pytorch
기반	Theano	Torch
개발사	Google	Facebook
그래프	정적	동적
실행 순서	모델의 전체 계산 그래프 정의 후 모델 실행	동작 중에 그래프 정의/조작
Deployment	배포 지향 app에 최적	부진했지만, 차이를 줄이는 중
GPU 할당	자동	<code>cuda.is_available()</code>

- Tensorflow: 상업적 모델 구현 및 배포, 상세한 구현
- PyTorch: 학문적 모델 구현, 라이브러리 이용, 편리한 사용

감사합니다